

Lambda Functions

Funzioni anonime di prima classe



Lambda functions

A block of code that you can pass around as so it can be executed later, once or multiple times.

Lambda syntax:

(parameters list) -> { lambda body }

- small, one or two line functions
- A lambda expression is an anonymous function, and it is **not associated with a class**.
- Reduce verbosity caused by anonymous classes and need for inner classes

- Alonzo Church in his invention of the lambda calculus in 1936.
- Lisp 1958
- Supported in C#, JavaScript, Python, Ruby, C++11

Lambda Syntax

```
//argument list
() -> System.out.println("Hello Lambda") ;
() -> {throw new RuntimeException()}
x -> x+10;
(int x, int y) -> { return x + y; };
(String x, String y) -> x.length() - y.length();
(x, y) -> x.length() - y.length();

//single expressions
x -> {x>=0 ? True: False}
()-> 42

//list of statements
(String x) -> {
    listA.add(x);
    listB.remove(x);
    return listB.size();
}
```

How are they different from Java methods?

- Argument type deduction
- Local variable capturing
- Omission of brackets and return statement
- Return type deduction

Vedi VSCode lambda/intro

Functional Interfaces

- An interface that has only one abstract method.
 - Before Java 8 this was obvious, only one method.
- Previously known as SAM's, Single Abstract Methods
- Were used in java.lang.Runnable, java.awt.event.ActionListener, java.util.Comparator, java.util.concurrent.Callable
- Java 8 introduced Default methods...
- Now use **@FunctionalInterface** annotation (may be omitted)
- Generates compiler error when there is more than one abstract method

Package java.util.function

- Well defined set of general purpose functional interfaces types.
 - All have only one abstract method.
- Lambda expressions can be used wherever these types are referenced.

```
new Thread(() -> System.out.println("Hello World!")).start();
```
- Inject functionality into methods, the same way we inject values into methods!
- Used extensively in the Java class libraries. Especially with the Stream API.
- Variations on number of arguments and specific for primitive types.

Functional Interfaces

- Attributing a lambda expression to a variable of Functional Interface Type
- The lambda expression provides the implementation of the abstract method.
- Returning a lambda expression is also possible

Example

```
@FunctionalInterface
public interface Runnable {
    public void run();
}

Runnable r = () ->
    System.out.println("Hello World!");
new Thread(r).start();

//functional interface implementation
List<String> strs = ...;
Collections.sort(strs, (s1, s2) ->
    Integer.compare(s1.length(), s2.length()));

new Thread(() -> {
    connectToService();
    sendNotification();
}).start();
```

Vedi vscode lambda/functionalInterfaces

Package java.util.function

- Predicate<T> - a boolean-valued property of an object. [boolean test()]
- Consumer<T> - an action to be performed on an object. [void accept(T)]
- Function<T,R> - a function transforming a T to a R. [R apply(T)]
- Supplier<T> - provide an instance of a T (such as a factory) [T get()]
- UnaryOperator<T> - a function from T to T. [T apply(T)]
- BinaryOperator<T> - a function from (T,T) to T [T apply(T, T)]
- More meaningful type names (vedi Oracle online docs)

Vedi esempio su VSCode: lambda/syntax

Method & Constructors References

Treating an existing method as an instance of a Functional Interface

- Object oriented way of attributing a method to a variable
- :: operator

```
class Person {
    private String name;
    private int age;

    public int getAge() {return this.age;}
    public String getName() {return this.name;}
}

Person[] people = ...;
Comparator<Person> byName = Comparator.comparing(Person::getName);
Arrays.sort(people, byName);
```

Kind of Method References

There are four kinds of method references:

Kind	Syntax	Examples
Reference to a static method	ContainingClass::staticMethodName	Person::compareByAge MethodReferencesExamples::appendStrings
Reference to an instance method of a particular object	containingObject::instanceMethodName	myComparisonProvider::compareByName myApp::appendStrings2
Reference to an instance method of an arbitrary object of a particular type	ContainingType::methodName	String::compareToIgnoreCase String::concat
Reference to a constructor	ClassName::new	HashSet::new

Kind of Method References

The lambda code block is executed in the context it references to...

1) To Static methods:

```
(args) -> ClassName.staticMethod(args)
ClassName::staticMethod
Consumer<Integer> b1 = System::exit;
```

2) To a generic instance's methods

```
(args) -> .instanceMethod(args)
ClassName::instanceMethod
Consumer<String[]> b2 = Arrays::sort;
```

3) To a specific instance's method

```
(args) -> ObjectName.instanceMethod(args)
ObjectName::instanceMethod
Consumer<String> b3 = myObj::doSomething;
```

4) To a constructor method

```
Factory<List<String>> f = () -> return new ArrayList<String>();
Factory<List<String>> f = ArrayList<String>::new;
```

f references a factory method. It can be passed around as parameter value...

External Variables

Lambda expressions can refer to variables from the surrounding scope.

```
class DataProcessor {
    private int currentValue;
    public void process(int par) {
        int n = 1000; // eff. Final
        DataSet myData = myFactory.getDataSet(n++);
        myData.forEach(d -> d.use(par, n, this.currentValue++));
    }
}
```

- Static and instance variable are ok. Only local variables must be **Effectively Final**

Effectively final: assigned only once, even if not explicitly declared final.

- 'this' refers to the enclosing object, not the lambda itself.
- Remember the Lambda is not associated with a class, therefore there can be no 'this' for a lambda.

Why Default Methods?

Add **default behaviours to interfaces**

Java 8 has lambda expressions.

```
List<?> list = ...
```

```
list.forEach( /* lambda code goes here */ );
```

```
@FunctionalInterface
public interface Iterable {
    default void forEach(Consumer<? super T> action) {
        for (T t : this) {
            action.accept(t);
        }
    }
}
```

There was a problem: can't use them...

Changing existing interfaces (java.util.List, java.util.Collection) would break their current library implementations

Solution: Java 8 introduce **default methods!** Existing libraries inherit the default implementation...

forEach(consumer<T>) is one default method in new Iterable<T> functional interface

New default interface methods in Java 8

Interface	Description
<code>Iterable.forEach(Consumer c)</code>	<code>myList.forEach(System.out::println);</code>
<code>Collection.removeIf(Predicate p)</code>	<code>myList.removeIf(s -> s.length() == 0);</code>
<code>List.replaceAll(UnaryOperator o)</code>	<code>myList.replaceAll(String::toUpperCase);</code>
<code>List.sort(Comparator c)</code>	<code>myList.sort((x, y) -> x.length() - y.length());</code>

Replaces `Collections.sort(List l, Comparator c)`

Default methods can be seen as a bridge between lambdas and existing JDK libraries.

Multiple inheritance in Java 8 ! Defaults can cause conflict of behaviour...

Conflict resolution

// Java program to demonstrate the case
// when two interfaces are overridden

```
// Creating Interface One
interface GfG {
    public default void display()
    {
        System.out.println("GEEKSFORGEEKS");
    }
}
```

```
// Creating Interface Two
interface gfg {
    public default void display()
    {
        System.out.println("geeksforgeeks");
    }
}
```

```
public class InterfaceExample implements GfG, gfg {

    // Interfaces are Overridden
    public void display() {
        GfG.super.display();
        gfg.super.display();
    }

    public static void main(String args[]) {
        InterfaceExample obj = new InterfaceExample();
        obj.display();
    }
}
```

Lambdas usages

- Look where methods are very similar except for 1 or 2 lines of code.
- To use streams for filtering, mapping, finding, matching, etc.
- Keep lambda expressions short for clarity, or write a helper method and call it in the lambda

Lambdas summary

- CPU's aren't getting much faster, but we are getting more cores to use.
- Inject functionality into methods, the same way we inject values into methods.
- Functional interfaces are interfaces that have only one abstract method.
- Lambdas can be used where the type is a functional interface.
- Use lambdas as method parameters or assign them to variables.
- The `java.util.function` package already gives us a few functional interfaces out of the box.
- Method and constructor references as shorthands notation, `::` (double colon)
- There are a few new methods that can use lambdas in Java 8, e.g. `forEach()`

10 min. break



Java Streams

filter/map/reduce for Java



What are streams?

- Streams give us functional blocks to better process collections of data
- We can chain these blocks together to process collections of data.
- Streams aren't another data structure (is an Interface).
- Streams can process an infinite list of data.
- Streams use internal iteration, meaning we don't have to code external iteration, the "how".
- Streams support functional programming style inside the imperative Java programming language.

Functional Programming

Imperative

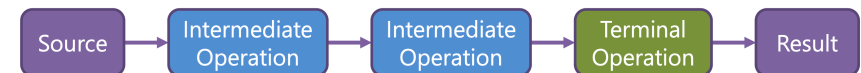
- Values associated with names can be changed, i.e. mutable state.
- The order of execution is defined as a contract.
- Repetition is external and explicit, we are responsible for the "how".

Functional (Declarative)

- Values associated with names are set once and cannot be changed, i.e. immutable state.
- Order of execution is not defined (is like in an expression evaluation)
- Repetition is through the use of recursion or internal repetition, the "how" is hidden away.

Structure of a Stream

- A stream consists of 3 types of things
 1. A source
 2. Zero or more intermediate operations
 3. A terminal operation



```
result = albums.stream()
    .filter(track -> track.getLength() > 300)
    .map(Track::getName)
    .collect(Collectors.toSet());
```

How they work

- The pipeline is only evaluated when the terminal operation is called.
- The terminal operations pulls the data, the source doesn't push it.
- Uses the stream characteristics to help identify optimisations.
- This allows intermediate operations to be merged or parallelised (fork/join)
- Avoiding multiple redundant passes on data
 - Short-circuit operations
 - Lazy evaluation
- The stream takes care of the "how".
- The stream is traversed only once
- Can be infinite

Creating Streams

Collection Interface `.stream()`

- Provides a sequential stream of elements in the collection.
- `parallelStream()` Provides a parallel stream of elements in the collection.
- Uses the fork-join framework for implementation.
- Only Collection can provide a parallel stream directly.

Arrays Class `.stream()`

- Array is a collection of data, so logical to be able to create a stream.
- Provides a sequential stream.
- Overloaded methods for different types, int, double, long, Object

Stream Interface `.builder()` (builder Pattern...)

`static <T> Stream.Builder<T> builder()` Returns a Stream Builder.

`void accept(T t), Stream add(T t)` Adds an element for later build.

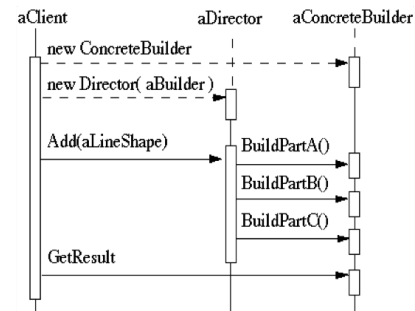
`Stream<T> build()` Builds the stream, no more adds...

Builder (creational design pattern)

Separate construction of a complex object from its representation

So same construction process can create different representations

Ex. Different representations (file formats) for complex documents (pdf, rtf, docx, etc...)



Stream sources

- Streams from values ("of" method)
 - `Stream<Integer> digits = Stream.of( 3, 1, 4, 1, 5, 9);`
 - `Stream<String> words = Stream.of( "Mary", "had", "a", "little", "lamb");`
- Empty streams
 - `Stream<Student> ss = Stream.empty();`
- Streams from functions
 - `Random random = new Random();`
 - `Stream<Integer> randomNumbers = Stream.generate(random::nextInt);`
- Streams from any array of objects
 - `Integer[] digitArray = { 3, 1, 4, 1, 5, 9 };`
 - `Stream<Integer> digitStream = Stream.of(digitArray);`
- Streams from collections
 - `Set<Student> set = new LinkedHashSet<>();`
 - `Stream<Student> stream = set.stream();`
- Streams from files (find, list, lines, walk).
 - `Stream<String> stream = Files.lines(path);`
- From other streams
 - `Stream newStream = Stream.concat(stream, randomNumbers);`

Primitive and infinite Streams

- `IntStream`, `DoubleStream`, `LongStream`

primitive data specialisations of the `Stream` interface.

`range(int, int)`, `rangeClosed(int, int)`

A stream from a start to an end value (exclusive or inclusive)

`generate(IntSupplier)`, `iterate(int, IntUnaryOperator)`

- An infinite stream created by a given `Supplier`.
- `iterate` uses a seed to start the stream and an induction.

Stream.iterate

static `<T> Stream<T> iterate (T seed, Predicate<? super T> hasNext, UnaryOperator<T> next)`

- Returns a sequential ordered `Stream`
- iteratively apply the given `next` function to an initial element
- The stream terminates as soon as the `hasNext` predicate returns false.
- produce the same sequence of elements as the for-loop:

```
for (T index=seed; hasNext.test(index); index = next.apply(index)) {...}
```

Example stream creations

<code>Stream.of(1, 2, 3)</code>	A stream containing the given elements. You can also pass an array.
<code>Collection<String> coll = ...;</code> <code>coll.stream()</code>	A stream containing the elements of a collection.
<code>Files.lines(path)</code>	A stream of the lines in the file with the given path. Use a try-with-resources statement to ensure that the underlying file is closed.
<code>Stream.generate(() -> 1)</code>	An infinite stream of ones
<code>Stream.iterate(0, n -> n + 1)</code>	An infinite stream of Integer values
<code>IntStream.range(0, 100)</code>	An <code>IntStream</code> of int values between 0 (inclusive) and 100 (exclusive)
<code>Random generator = new Random();</code> <code>generator.ints(0, 100)</code>	An infinite stream of random int values drawn from a random generator
<code>"Hello".codePoints()</code>	An <code>IntStream</code> of code points of a string

Demo su VSCode

streams/StreamSources



Parallel & Sequential

```
java.util.stream
List<Student> students = ...;
Stream stream = students.stream(); // sequential version

// parallel version
Stream parallelStream = students.parallelStream();
```

- Streams can switch between sequential and parallel, but all processing is either done sequential or parallel, last call wins.

```
List<Person> persons = ...;
List<Student> students = persons.stream()
    .parallel()
    .filter(p -> p.getAge() > 18)
    .sequential()
    .map(Student::new)
    .collect(Collectors.toCollection(ArrayList::new));
```

Intermediate Operations

- Most operations take a parameter that describes its behaviour, the “what”.
- Typically using lambda expressions.
- Must be non-interfering: stateless

The stream elements iteration is hidden in the stream library implementation

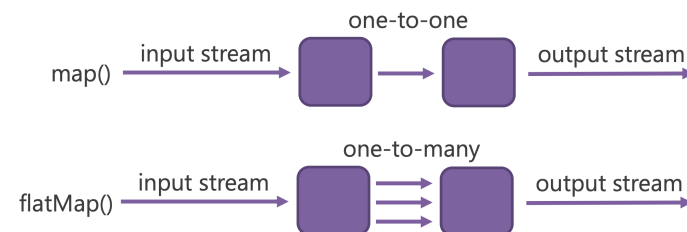
```
List<Person> persons = ...;
Stream<Person> tenPersonsOver18 = persons.stream()
    .filter(p -> p.getAge() > 18)
    .limit(10);
```

Intermediate Operations

- `.filter(Predicate)` - excludes all elements that don't match a Predicate
- `.map(Function)` - perform transformation of elements using a Function
- `.mapToInt`, `.mapToDouble`, `.mapToLong` - Like `map()`, but producing streams of primitives
- `.flatMap(Function)` - transform each element into a substream of zero or more elements
- `.peek(Consumer)` - performs action on each element, useful for debugging (non terminal forEach)
- `.distinct` - excludes all duplicate elements (based on `equals(T)`)
- `.sorted(Comparator)` - return an ordered stream. With no arguments sorts by natural order.
- `.limit(long)` - Return a stream that only contains the first n elements of the input stream
- `.skip(long)` - Returns a stream that skips the first n elements of the input stream

Map() and FlatMap()

- Map values from a stream either as one-to-one or one-to-many, but still only produce one stream.



See example [streams/operations/mapping on github](#)

Terminal Operations

Terminates the pipeline of the operations on the stream.

- Only at this point is any processing performed.
- This allows for optimisation of the pipeline:
- Lazy evaluation
- Merged/fused operations
- Elimination of redundant operations
- Parallel execution
- Generates an explicit result or a side effect.

```
List<Person> persons = ..;  
List<Student> students = persons.stream()  
    .filter(p -> p.getAge() > 18)  
    .map(Student::getAge)  
    .collect(Collectors.toList());
```

Collectors

We can accumulate *Stream* elements into a *List* by using:

- *Stream.collect(Collectors.toList())*: Since Java 8
- *Stream.toList()*: Since Java 16

```
List<String> list = people.stream()  
    .map(Person::getName)  
    .collect(Collectors.toList());  
  
// Accumulate names into a TreeSet  
Set<String> set = people.stream()  
    .map(Person::getName)  
    .collect(Collectors.toCollection(TreeSet::new));  
  
// Convert elements to strings and concatenate them, separated by commas  
String joined = things.stream()  
    .map(Object::toString)  
    .collect(Collectors.joining(", "));  
  
// Compute sum of salaries of employee  
int total = employees.stream()  
    .collect(Collectors.summingInt(Employee::getSalary));
```

Demo su GitHub

streams/Refactor

streams/RefactorExercise



Grouping/Partitioning Collectors

```
// Group employees by department  
Map<Department, List<Employee>> byDept = employees.stream()  
    .collect(Collectors.groupingBy(Employee::getDepartment));  
  
// Compute sum of salaries by department  
Map<Department, Integer> totalByDept = employees.stream()  
    .collect(Collectors.groupingBy(Employee::getDepartment,  
        Collectors.summingInt(Employee::getSalary)));  
  
// Partition students into passing and failing  
Map<Boolean, List<Student>> passingFailing = students.stream()  
    .collect(Collectors.partitioningBy(s -> s.getGrade() >= PASS_THRESHOLD));
```

Demo su GitHub

streams/Refactor

streams/RefactorExercise



Optional

- `java.util.Optional<T>`
 - a new class to help eliminate `NullPointerException`'s.
- Terminal operations like `min()`, `max()`, may not return a direct result, suppose the input stream is empty?
- It is a container for an object reference (`null`, or real object).
 - It has either 0 or 1 elements, but is never `null`.
- Doesn't stop developers from returning `null`, but an `Optional` tells you what to do (in case), rather than having to check.

Creating an Optional

<code><T> Optional<T> empty()</code>	Returns an empty <code>Optional</code> .
<code><T> Optional<T> of (T value):</code>	Returns an <code>Optional</code> containing the specified value. If the specified value is <code>null</code> , it throws a <code>NullPointerException</code> .
<code><T> Optional<T> ofNullable (T value)</code>	Same as above, but if the specified value is <code>null</code> , it returns an empty <code>Optional</code> .

Getting Data from an Optional

<code><T> get()</code>	Returns the value, or throws <code>NoSuchElementException</code> if value is empty.
<code>boolean isPresent()</code>	will return true If a value is present, false otherwise
<code><T> orElse(T defaultValue)</code>	Returns the default value if value is empty.
<code><T> orElseGet(Supplier<? Extends T> defaultSupplier)</code>	Same as above, but supplier gives the value.
<code><X extends Throwable> T orElseThrow(Supplier<? Extends T> exceptionSupplier)</code>	If empty throw the exception from the supplier.
<code>void ifPresent(consumer<T>)</code>	execute a block of code with the value, if it is present.

Demo su VSCode

streams/Optional



Folding to a Single Result: reduce

- The `collect` function isn't the only option.
- `reduce(BinaryOperator accumulator)` $(T, T) \rightarrow T$ where T is the type of the stream objects
Also called folding in Functional Programming.
- Performs a reduction on the stream using the `BinaryOperator`.
- The `accumulator` takes a partial result and the next element and returns a new partial result as an `Optional`.
- Two other versions
 - One that takes an initial value (a "Identity" value object, with respect to the applied folding).
 - One that takes an initial value and `BiFunction`. $(T, U) \rightarrow U$ to generate a result of a different type U
`<U> U reduce(U identity, BiFunction<U, ? super T, U> accumulator, BinaryOperator<U> combiner);`

Terminal Operations: collections and numeric results

Interface	Description
<code>collect(Collector*)</code>	Performs a mutable reduction on a stream
<code>toArray()</code>	Returns an array containing the elements of the stream
<code>count()</code>	Returns how many elements are in the stream
<code>max(Comparator)</code>	The maximum value element of the stream, returns an <code>Optional</code>
<code>min(Comparator)</code>	The minimum value element of the stream, returns an <code>Optional</code>
<code>average()</code>	Return the arithmetic mean of the (primitive only) stream, returns an <code>Optional</code>
<code>sum()</code>	Returns the sum of the stream elements
<code>reduce(BinaryOperator)</code>	applies an <i>associative</i> accumulation function to the stream and reduces it to an <code>Optional</code>

* There are a lot of built-in Collectors: `toList()`, `toSet()`, `toMap(Function, Function)`, etc...

Terminal Operations

Interface	Description
<code>findFirst(Predicate)</code>	The (Optional) first element that matches predicate
<code>findAny(Predicate)</code>	Like <code>findFirst()</code> , but for a parallel stream
<code>allMatch(Predicate)</code>	True if All elements in the stream match predicate
<code>anyMatch(Predicate)</code>	True if Any element in the stream matches predicate
<code>noneMatch(Predicate)</code>	True if No elements match the predicate
<code>forEach(Consumer)</code>	Performs an action on each element (ret. void)
<code>forEachOrdered(Consumer)</code>	Like above, but ensures order is respected when used for parallel stream

Demo su VSCode

streamLibro



Altri esempi da vedere a casa

- There are 5+21 exercises to complete at home for practice.
- There is a template source file that you should use to create the answers to the exercises called Test.java under exercises package in the test folder.
- To simplify things the lists and maps you need for the exercises have already been created.
- You just need to focus on the code to solve the exercise.
- The solutions are in the Test.java file in the solutions package, but try all the exercises first before taking a peak.

10 min. break



Fundamental Challenges

Testing for correctness



Outline

- Principles and practices for dealing with fundamental challenges:
 - Requirements can change: apply the right **Process**!
 - Software is intrinsically complex: **Design** for easy maintenance!
 - Defects are inevitable: **Test** the software!
- These challenges have implications for what to build and how to build it

Software is Intrinsically Complex

Why?

- Let us say software is complex if it is hard to understand and modify
- Accidental (or Incidental) complexity is tied to an implementation
 - It can be removed by cleaning up the code, changing the language, etc.
- Intrinsic complexity remains even if all accidental complexity is removed
 - It can be managed by designing run-time behaviour more predictable

Intrinsic Complexity: Run-Time Behaviour is Invisible

- The static (compile time) source text is visible
 - We can read and write code
- But, the dynamic (run-time) behaviour is invisible
 - We have to imagine what the code does when it runs
- The primary source of intrinsic complexity
 - The predictability (or not) of system behaviour from the source code
 - In short, the invisibility of behaviour is the primary source

Does the following loop terminate cleanly?

- `do j++; while ( a[j] < pivot );`

Example: Clean Loop Termination

- Does the following loop terminate cleanly?
 - `do j++; while ( a[j] < pivot );`
- Answer: depends on the values of j, pivot, and the array elements
 - For clean termination, the loop must reach `a[k] >= pivot`
- Conclusion
 - The same code can have a potentially infinite number of outcomes

Dealing with Complexity

- Avoid Accidental (or Incidental) complexity
 - Use the right languages, frameworks, and tools for the job
 - Avoid technical debt, which accumulates, and makes code brittle
 - Technical debt is the conceptual cost of cleaning up a “quick fix”
- Manage Intrinsic complexity
 - **Design modular systems**, with relatively independent parts
 - Low coupling, high cohesion
 - Refactor to improve the design
 - Apply well-known design patterns

Defects are Inevitable

Why?

- From George Miller's Experiments in the 1950s
 - People can keep track of about 7 units: bits, words, colors, tastes, tomes
 - Beyond about 7 chunks of information, confusion and errors set in
- To err is human

Faults versus Failures

- Faults are in the static source code or in documents
 - Defect, bug, and anomaly are synonyms for faults
- Failures are triggered at run time (dynamically)
 - A failure is unexpected behavior, with crashes being dramatic failures
- Example: When an app crashes some of the time
 - There must be a fault or faults in the app's code
 - But, failures occur only some of the time; e.g., when there is a crash

Terminology

Validazione (testing)

- verifica del "cosa" fai: "Am I building the right product?"
- Controllo (eseguendo prove) se le funzionalità corrispondono a quelle specificate nei requisiti
- È il punto di vista dell'utente/utilizzatore: correttezza che funziona?
 - **Black-box** tests are based on inputs and outputs; no need for the code
 - **White-box** tests are based on knowledge of the code

Verifica (formal analysis)

- Verifica del "come" lo fai: "Am I building the product right?"
- Controllo se i meccanismi interni sono corretti per lo scopo
- È il punto di vista dall'interno, dello sviluppatore
 - Es.: testo il metodo `int somma(a,b) { return a*b-1; }` con `a=3` e `b=2`
 - Supera il test, ma non la verifica...

Formal Verification

- I meccanismi interni sono corretti ?
- Il testing non dà la certezza che funzioneranno sempre
- Riduco la complessità eccessiva passando dal codice ad un suo "modello formale"
- Provo con certezza matematica la sussistenza o meno di proprietà di correttezza sul modello (ad. Es. assenza di deadlock, di loop infiniti, etc...)
 1. Con "Proof Assistant" tools (PVS, Coq, HOL, Isabelle, etc...)
 2. Con "Model Checker" tools (SPIN, NuSMV, PathFinder, etc...) che cercano nel modello contro-esempi che falsificano la proprietà

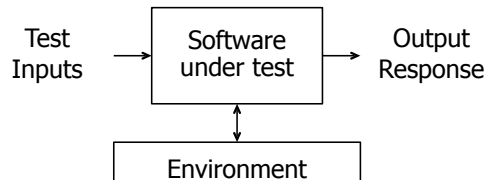
Una astrazione (modello) sbagliata anche di pochissimo può alterare dettagli importanti del codice, e le proprietà dimostrate non sono più trasferibili dal modello (al software, e quindi) alla realtà.

Testing

- Eseguire test permette di controllare se il **codice soddisfa le specifiche del cliente: i test si possono scrivere a partire dalla specifica dei requisiti, dal codice o da entrambi**
- Testare in modo ampio permette di ottenere una valutazione sulla **qualità** del sistema
 - I test devono essere indipendenti fra loro ed auto valutarsi
- **Responsabilità**: i compiti sono suddivisi uno per metodo, o per classe, questo permette di ottenere **coesione** del codice e che i test di correttezza di una data funzionalità si possano concentrare su un singolo metodo o classe
 - **Principio di Singola Responsabilità (SRP)**. La singola responsabilità è **cruciale** per la buona comprensione, il riuso, l'ereditarietà (impedisce ambiguità e sovrapposizioni tra gli ADT)

Testing Overview

- 1. Decide on the Software Under Test and white box or black box
- 2. Set up its Environment so the code can be run repeatably
- 3. Select a set of test inputs based on a criterion for good tests
- 4. Evaluate the outputs with the intent of finding bugs



Testing is a Form of Sampling

- We have two typically infinite sets, called populations or domains
 - Inputs: integer inputs form an infinite set
 - Executions: with `for(i=1; i<N; i++){...}`, N can be any integer
- Choose tests so they faithfully represent the population
 - Black-box tests are chosen to **sample the input domain**, as we'll see
 - White-box tests are chosen to **sample executions**, as we'll see

Select Tests for Coverage

- How will test inputs be selected? What is a good test set?
- How much tests are enough?
- **Coverage** is the fraction of constructs touched by a set of tests
- A **coverage metric** is the criterion to quantitatively measure the coverage
- Select white-box tests for **code coverage**
 - Statement coverage is a typical code-coverage criterion
 - Decision or Branch coverage is a stronger criterion
- Select black-box tests for **input-domain coverage**
 - Select test inputs so they are a representative sample of all inputs
 - Test all Combinatorial Interactions of a degree

Testing Cannot Prove the Absence of Bugs

- Example: There had been 300 successful test runs of the guidance system
 - Shortly after launch, the Atlas rocket wobbled off course
 - 293 seconds after launch, it was destroyed by the safety officer
- What happened?
 - Signal contact with the ground was lost
 - The engineers had anticipated this possibility, but the code had a bug
- Moral of the Story: 300 test runs did not uncover the bug

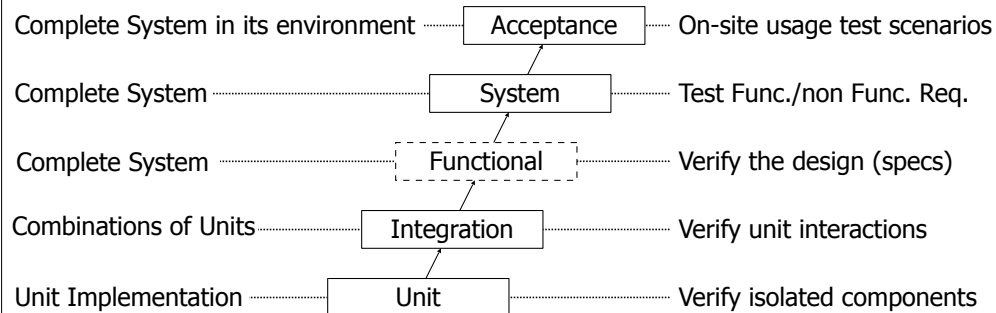
Testing Terminology

- **Unit testing** is named after units of implementation (single function, single class in OOP)
- Regression testing runs all **known tests** to verify that nothing broke
- A **test suite** is a set of test inputs (—> test case instances/items)
- A **System Verification Group** is a group dedicated to testing

Defining a Test Strategy

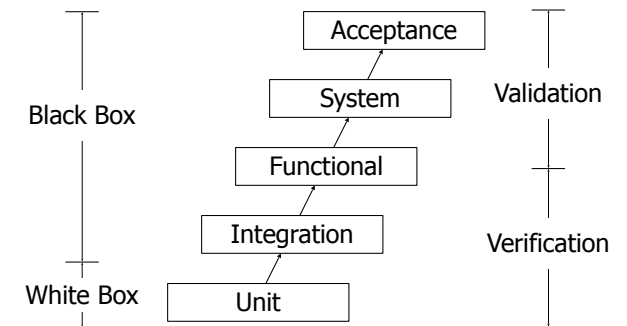
- What is the software under test?
 - **White-box** tests are based on knowledge of the code
 - **Black-box** tests are based only on inputs and outputs
- What does the software need from its **environment** to before it can run?
 - The software is typically not self-standing. It needs **values or functions**.
- How will test inputs be selected? What is a good test set?
- How will the output be evaluated? That is, did a test pass or fail?

Software Under Test and Levels of Testing



Primary Focus of the Levels of Testing

- Il WBT è possibile solo sulle Unità
- le Unit ammettono anche il BBT
- Fino al livello funzionale, ha senso parlare di verifica di correttezza rispetto a specifiche espresse in modo formale
- La validazione (testing) è possibile in tutti i livelli



Testing

- Validation tests tend to be black-box
- Verification tests tend to be white-box, but **black-box tests can also be used**
- With agile methods, developers test as they code
- Continuous deployment relies heavily on **automated testing**
- Manual steps add delay and potentially errors
- **Test can exist before code**
 - Test Driven Development (**TDD**) and Behaviour Driven Development (**BDD**) write tests before writing unit code

Case Study: Large Project, Strong Focus on Testing

- Planning phase
 - Extensive customer involvement
 - Highly skilled team with deep experience with similar projects
- Sprints measured in weeks
 - Feature sets developed in parallel by subteams
 - Nightly builds and regression tests
 - Intense focus on early detection of correctness and performance issues

Develop Feature Sets in Parallel

Deliver code and unit tests and perform sanity functional tests.

Code &
Unit Tests

Deliver Code and Automated Functional Tests

Also, start Interoperability Testing with third party systems

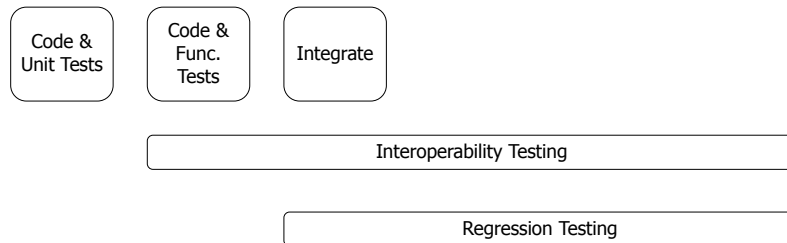
Code &
Unit Tests

Code &
Func.
Tests

Interoperability Testing

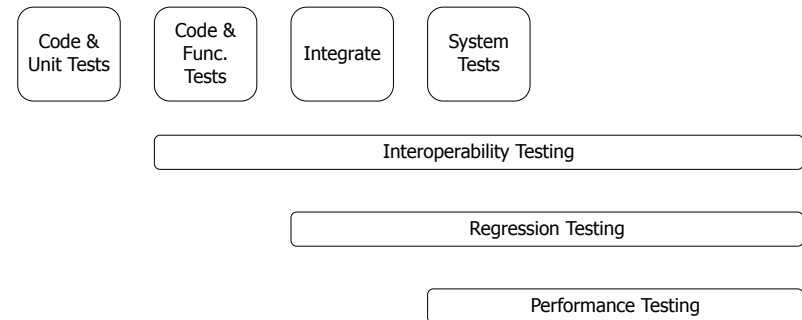
Integration: Approve Features for SV

Start Regression Testing (SV stands for System Verification)



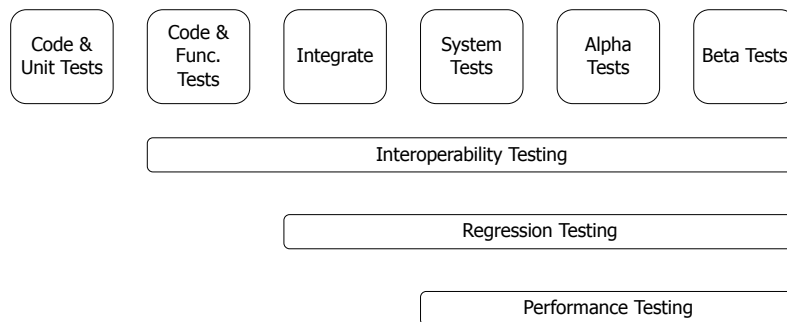
SystemTest: Test Features for Failure Scenarios

Start Performance Testing

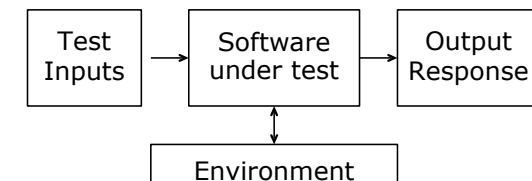


Internal Alpha and External Beta Tests

Alpha tests are trials within the company, beta tests are customer trials



Testing Context : what is the environment?



The Role of the Environment

- The environment provides the software under test what it needs to run
 - Often, the software under test is **not self-standing**
- Example: `day()` converts an object of class `Date` to a day of the year
 - `int day(Date d) { ... if( isleap(d.year) { ... } ... }`
- For the code for `day()` to run, the environment must supply
 - An input `d`, a declaration of `Date`, and a function `isleap()`

Stabilize the Environment

- What does it mean to stabilize the environment?
 - Provide consistent values to make tests repeatable
 - Provide a controlled setting for testing expected and unexpected behavior
- How?
 - Simulate the interfaces and the messages across them
 - Interfaces include software APIs, file systems, communications
 - Human inputs can be a challenge; e.g., gestures that cannot be recorded

Drivers and Stubs

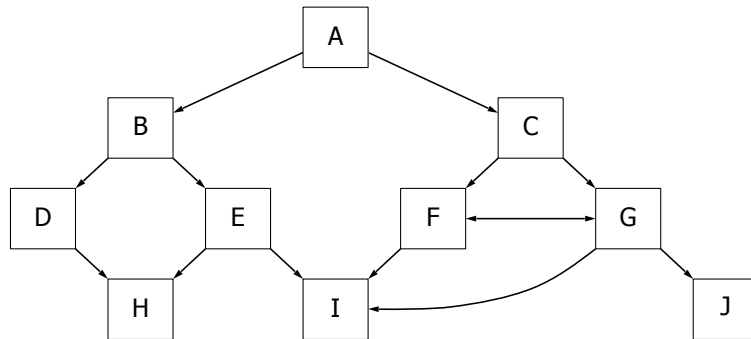
- A driver for module `N`
 - Simulates an environment for `N`
 - May provide stubs for modules or functions used by `N`
- A stub for module `M`
 - Has the same interface as `M`
 - Simulates the functionality provided by `M`
 - May return dummy or pre-programmed values
 - Usually called “mock objects”

System Integration and Testing

- Start with unit-tested modules
 - Then, integration errors are likely to be due to module interactions
- Integrate the whole system all at once, if possible
 - Then, the environment is only for external services
- Incremental integration builds up the system in stages
 - Top-down integration requires stubs for used modules
 - Bottom-up integration requires drivers

What order would you integrate and test these modules?

An edge from M to N means that module M uses module N

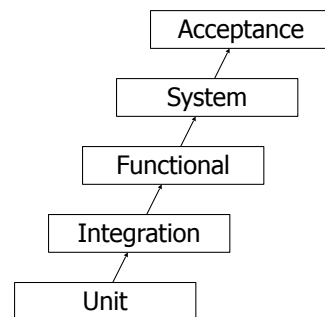


Incremental Integration

- Bottom-up integration
 - Any order in which used modules appear before using modules
- Top-down integration
 - Using modules appear before used modules
- Modules that are in a cycle must be integrated and tested together

Test Automation

- Continuous deployment relies heavily on automated testing
- Manual steps add delay and potentially errors
- Example test automation tools:
 - JUnit
 - TestNG
- Example test generation tools
 - Randoop, EvoSuite, Ttuples



JUnit

- Integrazione con tool e IDE diffusi (Eclipse, IntelliJ, VS Code)
- Scrivo classi per il test, in cui inserisco metodi annotati con **@Test**
- Ogni metodo di test è trovato ed eseguito automaticamente dal framework (usando la riflessione computazionale)
- Ogni metodo viene eseguito in una nuova istanza della classe di test
 - per avere ripetizioni indipendenti ed evitare effetti collaterali
- Si usano **asserzioni** per automatizzare la valutazione dell'esito del test

Assertions

- La classe Assert fornisce i metodi usati per valutare il test
- `assertTrue(boolean condition)` valuta se `condition` è true
- `assertEquals(a, b)` verifica se due oggetti sono uguali
- Altri metodi assert
 - `assertNull(Object object)`
 - `assertSame(expected, actual)`
 - `assertTrue(boolean condition)`
 - `assertFalse(boolean condition)`
 - `fail(String message)` //termina il test, utile per testare le routines di eccezione
- I metodi assert registrano fallimenti o successi e li riportano
- Quando si registra un fallimento, l'esecuzione del metodo di test corrente viene interrotta, ma si prosegue con gli altri

```
import org.junit.Test;
import static org.junit.Assert.*;

public class TestCalc { // classe di test

    @Test
    public void testAdd() { // l'annotazione indica il test
        Calculator c = new Calculator(); // crea istanza
        double result = c.add(10, 50); // chiama metodo da testare
        assertEquals(60, result, 0); // controlla il risultato e
        // genera eccezione se result != 60
    }
}
```

Esempio calculator su vscode...

Test suites

- Una selezione di classi di test può confluire in una test suite
- Posso raggruppare allo stesso modo alcune test suites in una test suites di livello superiore
- altrimenti posso eseguire singoli metodi, singole classi o tutto.

```
@RunWith(Suite.class) // Runner: questa è una test suite
@Suite.SuiteClasses({ConsumerTest.class, SommatoreTest.class})
public class TestSuiteA { }
```

```
@RunWith(Suite.class) // Runner: questa è una test suite
@Suite.SuiteClasses({CalculatorTest.class, SommatoreTest.class})
public class TestSuiteB { }
```

```
@RunWith(Suite.class) // Runner: questa è una test suite di suites
@Suite.SuiteClasses({ TestSuiteA.class, TestSuiteB.class })
public class MasterTestSuite { }
```

@Before e @After

- Un metodo annotato con @Before (setUp)
 - Viene chiamato prima di ogni @Test
 - Lo uso per inizializzare lo stato dell'istanza della classe di test
 - In modo che i tutti i test partano dalle stesse condizioni
- Un metodo @After (tearDown)
 - Per operazioni di chiusura o finali (clean)
 - Viene eseguito comunque sia andato il test

• Vedi esempio Calculator2...

@BeforeClass e @AfterClass

- @BeforeClass annota un metodo statico
 - Eseguito prima di iniziare il lancio dei @Test della classe
 - Utile per eseguire operazioni costose, una tantum
 - es. apertura connessione remota e query a un database
 - Essendo un metodo statico, può modificare solo attributi statici della classe
- Analogamente @AfterClass

• Esempio: Singleton su VSCODE

AssertJ

- AssertJ può essere usata assieme a Junit per definire asserzioni complesse con uno stile *fluent*
- Permette di scrivere asserzioni concatenate, migliorando la leggibilità

```
String hobbit = "frodo";
```

```
// JUnit Assertions
assertTrue(hobbit.startsWith("Fro"));
assertTrue(hobbit.endsWith("do"));
assertTrue(hobbit.equalsIgnoreCase("frodo"));
```

```
// AssertJ
assertThat(hobbit).startsWith("Fro").endsWith("do").isEqualToIgnoringCase("frodo");
```

Mock Objects (oggetti finti)

ConsumerIntegrationTest
SommatoreTest
Consumer.TestSumTwice_NoMockito

- Il test di unità deve testare la correttezza di **una singola classe** (o sarebbe un integration test...)
 - **simulo** il comportamento delle classi con cui collabora, con degli oggetti **finti**.
 - devo **isolare** la classe sotto test: lo sviluppo delle altre classi può essere incompleto (c'è solo l'interfaccia), non ancora testato o comunque errato
- creo un'istanza di una classe che implementa per finta l'interfaccia del vero collaboratore, oppure direttamente un'istanza di sottoclasse anonima
- Definisco il comportamento dei suoi metodi direttamente nel test, limitato a restituire un risultato predefinito per input predefiniti (comportamento da **stub**)

Tipologie di Test Doubles

- **Dummy** objects are passed around but never actually used. Usually they are just used to fill parameter lists.
- **Fake** objects actually have working implementations, but usually take some shortcut which makes them not suitable for production (an in memory database is a good example).
- **Stubs** provide canned answers to calls made during the test, usually not responding at all to anything outside what's programmed in for the test.
- **Spies** are stubs that also record some information based on how they were called. One form of this might be an email service that records how many messages it was sent.
- **Mock objects** can be programmed to know how many times and in what order functions should be called during testing.

Mock vs Stub

Gli stub servono a fornire dati di ritorno alle chiamate della classe testata:

Decido l'esito del test con asserzioni che esaminano lo stato della classe testata
Con mockito possono fare anche questo.

Mock sono oggetti finti creati per controllare il comportamento della classe sotto test: **le sue interazioni con il mock**
Tiene conto del **numero**, dell'**ordine** e dei valori nei **parametri** delle chiamate ricevute durante il test
Le asserzioni, e l'esito del test, esaminano lo stato del mock, non della classe testata.

BDD: Behaviour Driven Development (complementare al Test Driven Development)



Behaviour Driven Development (BDD)

Style of writing tests that uses `//given` `//when` `//then` comments as fundamental parts of your test methods.

The acceptance criteria validating the feature under test is a scenario, with the following structure:

- **Given:** the initial context at the beginning of the scenario, in one or more clauses; che specificano cosa va restituito dai metodi mock quando invocati
- **When:** the event that triggers the scenario; La classe testata viene usata e usa in cascata anche il mock object
- **Then:** the expected outcome, in one or more clauses: asserzioni sullo stato e/o sulle interazioni

Mockito per Java

Consumer.TestSumTwiceMockito su vscode

Libreria che semplifica la scrittura di test coi Mock objects

Non è più necessario implementare nei test classi finte

Vanno solo dichiarate e inizializzate col un comando: **initMock()**

```
@Mock
Collaborator mockObj;

//sintassi alternativa, senza annotazione
Collaborator mockObj = mock(Collaborator.class);
```

Mockito permette:

Di indicare direttamente durante l'esecuzione del test, quale valore restituire quando un metodo del mock viene invocato (**when/given**)

Di verificare se e con quali input un determinato metodo del mock sia stato chiamato durante l'esecuzione del test (**verify/then**): vero uso da Mock

MockitoBDD per Java

Il test valuta se, quante volte e con quali input i metodi dei mock sono stati chiamati durante l'esecuzione del test

```
Person person = mock(Person.class);
Seller police = mock(Police.class);
Bike bike = new bike();

//when
person.ride(bike);
person.ride(bike);

//then
then(person).should(times(2)).ride(bike);
then(person).shouldHaveNoMoreInteractions();
then(police).shouldHaveZeroInteractions();
```

Consumer.TestSumTwiceBDD

Behaviour Driven Development (BDD)

```
Seller seller = mock(Seller.class);
Shop shop = new Shop(seller);

public void shouldBuyBread()
//given
given(seller.askForBread()).willReturn(new Bread());
//when
Goods goods = shop.buyBread();
//then
assertThat(goods, containBread());
}
```

Test parametrici

Il metodo (o i metodi) della classe di test vengono ripetuti per ogni tupla di assegnamenti indicata in un array statico annotato @Parameters

```
@Parameters
public static Collection<Integer[]> getParam() {
    return Arrays.asList(new Integer[][] {
        // a, b, expected
        { 1, 1, 2 },
        { 3, 2, 5 },
        { 4, 3, 7 },
    });
}
```

Le tuple sono passate ad ogni nuova istanza del costruttore della classe di test

```
public MyTestClass(int a, int b, int expected) {...}
```

Vediamo un esempio: `Consumer.TestSumTwiceBDD`

Black-Box White-Box Testing



Comparison

Black-Box Testing	White-Box Testing
Tests are only dependent on the specification.	Tests are dependent on both the implementation and the specification.
Tests can be re-used if the code is updated, which may be to fix a fault or add new features.	Tests are generally invalidated by any changes to the code, and cannot be reused.
Tests can be developed before the code is written as they only require the specification.	Tests can only be developed after the code is written as they require both the specification and the executable code.
Tests do not ensure that all of the code has been executed. They may miss <i>errors of commission</i> (see Section 8.1.3).	Tests do not ensure that the code fully implements the specification. They miss <i>errors of omission</i> .
Few tools provide automated coverage measurement of any black-box tests.	Many tools provide automated coverage measurement of white-box tests.

Usage

- BBT and WBT generally used in succession to maximise coverage
- The degree of coverage is usually based on the quality requirements of the test item. For example, software that decides whether to sound an alarm for a hospital patient will have higher quality requirements than software that recommends movies to watch
- In order:
 - Basic BBT
 - Augment with extra BBT
 - Basic WBT
 - Augment with extra WBT
- And use repair-based testing after fixing software faults

Basic BBT

- **EP:** Use equivalence partitions to verify the basic operation of the software by ensuring that one representative of each partition is executed
- **BVA:** If the specification contains boundary values, use boundary value analysis to verify correct operation at the edges
- **CIT:** If the specification states different processing for different combinations of inputs, or there is a complex input model, with many multi-valued parameters, a combinatorial interaction test suite can exercise it effectively

Basic WBT

- **Measure** the statement and branch coverage for black-box tests
 - If required coverage not achieved, add white-box testing
 - To enhance quality of testing
- **SC**: Use statement coverage to ensure that 100% of the statements have been executed
- **BC**: Use branch coverage to ensure that 100% of the branches have been taken

Extra WBT

- **AP**: If code contains complex end-to-end paths, use all paths testing
- **DU-Pairs**: if code contains complex data usage patterns, then use du pair testing
 - Described later in this lecture
- **CC, DC, DCC, MCC, MCDC**: If code contains complex decisions, then use these forms of testing to ensure coverage of these
 - Described later in this lecture

Equivalence Partitions



Equivalence Partitions

- An Equivalence Partition is **a range of values for a parameter** for which the specification states **equivalent processing**
- Example:
 - boolean isNegative(int x)
 - Returns true if x is negative, otherwise false
- Two equivalence partitions for the parameter x can be identified:
 - Integer.MIN VALUE..-1
 - 0..Integer.MAX VALUE

Step 1. Analysis

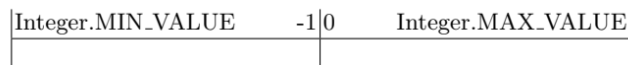
- Analyse the specification to identify the equivalence partitions
- Two stages:
 - first identify the **natural ranges** for each parameter
 - then identify the **specification-based ranges** (or equivalence partitions)
- Based on the principle of equivalent processing

Natural Ranges

- Natural ranges for common types
 - **byte** [Byte.MIN VALUE..Byte.MAX VALUE]
 - **short** [Short.MIN VALUE..Short.MAX VALUE]
 - **int** [Integer.MIN_VALUE..Integer.MAX_VALUE]
 - **long** [Long.MIN VALUE..Long.MAX VALUE]
 - **char** [Character.MIN VALUE..Character.MAX VALUE]
- For types with no natural ordering, each value is a separate range containing one value
 - **boolean** [true][false]
 - **enum Colour {Red, Blue, Green}** [Red][Blue][Green]
 - Treat numeric outputs with limited, specific values in the same way as enum's

EP Example

- Two equivalence partitions for the parameter x can be identified:
 - Integer.MIN VALUE..-1
 - 0..Integer.MAX_VALUE
- These specification-based ranges are called equivalence partitions
- According to the specification, any value in the partition is processed equivalently to any other value



Analysis/Parameters

- Methods (and functions) have **explicit and implicit** parameters
- Explicit parameters are passed in the method call
- Implicit parameters are not:
 - in C may be **global variables**
 - in Java may be **attributes**
- Both types of parameter must be considered in testing
- A complete specification should include all inputs and outputs.

Test Coverage Items (TCI)

- **Each partition** for each input and output **is a test coverage item**
- It is good practice to give each test coverage item (for each test item) a unique identifier
- It is often useful to use the prefix "EP-" for EP test coverage items
- So, for example, for two methods, you can have:
 - method1(): EP1, EP2, EP3, etc.
 - method2(): EP1, EP2, EP3, etc.
- Selected values used in the Test Cases

giveDiscount()

Status giveDiscount(long bonusPoints, boolean goldCustomer)

Inputs

bonusPoints: the number of bonusPoints the customer has accumulated

goldCustomer: true for a Gold Customer

Outputs

return value:

FULLPRICE if bonusPoints ≤ 120 and not a goldCustomer

FULLPRICE if bonusPoints ≤ 80 and a goldCustomer

DISCOUNT if bonusPoints > 120

DISCOUNT if bonusPoints > 80 and a goldCustomer

ERROR if any inputs are invalid (bonusPoints < 1)

Status is defined as follows:

```
enum Status { FULLPRICE, DISCOUNT, ERROR };
```

Equivalence Values

Parameter	Equivalence Partition	Equivalence Value
bonusPoints	Long.MIN_VALUE..0	-100
	1..80	40
	81..120	100
	121..Long.MAX_VALUE	200
goldCustomer	true	true
	false	false
Return Value	FULLPRICE	FULLPRICE
	DISCOUNT	DISCOUNT
	ERROR	ERROR

•

Test Coverage Items for giveDiscount()

TCI	Parameter	Equivalence Partition	Test Case
EP1*	bonusPoints	Long.MIN_VALUE..0	To be completed later
EP2		1..80	
EP3		81..120	
EP4		121..Long.MAX_VALUE	
EP5	goldCustomer	true	
EP6		false	
EP7	Return Value	FULLPRICE	
EP8		DISCOUNT	
EP9		ERROR	

Selecting Next Uncovered TCIs

TCI	Parameter	Equivalence Partition	Test Case
EP1*	bonusPoints	Long.MIN_VALUE..0	To be completed later
EP2		1..80	
EP3		81..120	
EP4		121..Long.MAX_VALUE	
EP5	goldCustomer	true	
EP6		false	
EP7	Return Value	FULLPRICE	
EP8		DISCOUNT	
EP9		ERROR	

EP Test Cases for giveDiscount()

ID	TCI Covered	Inputs		Exp. Results
		bonusPoints	goldCustomer	return value
T1.1	EP2,5,7	40	true	FULLPRICE
T1.2	EP3,6,[7]	100	false	FULLPRICE
T1.3	EP4,[6],8	200	false	DISCOUNT
T1.4*	EP1*,9	-100	false	ERROR

- Each **input error** test coverage item is tested separately
- **Minimising** the number of test cases can require multiple iterations
- Target – the maximum number of partitions of any input or output:
 - In this example it is 4 (for the bonusPoints parameter)
 - Not always achievable
- Combinations of input values are not considered
 - In this example, no test for bonusPoints equal to 40 with goldCustomer both true and false

Reviewing the Test Coverage Items

- Once complete, check that
 - every TCI is covered by at least one test
 - no duplicate (redundant) tests

ID	TCI Covered	Inputs		Exp. Results
		bonusPoints	goldCustomer	return value
T1.1	EP2,5,7	40	true	FULLPRICE
T1.2	EP3,6,[7]	100	false	FULLPRICE
T1.3	EP4,[6],8	200	false	DISCOUNT
T1.4*	EP1*,9	-100	false	ERROR

TCI	Parameter	Equivalence Partition	Test Case
EP1*	bonusPoints	Long.MIN_VALUE..0	T1.4
EP2		1..80	T1.1
EP3		81..120	T1.2
EP4		121..Long.MAX_VALUE	T1.3
EP5	goldCustomer	true	T1.1
EP6		false	T1.2
EP7	Return Value	FULLPRICE	T1.1
EP8		DISCOUNT	T1.3
EP9		ERROR	T1.4

Fault Model

- The equivalence partition fault model is where **entire ranges of values are not processed correctly**
- These faults can be associated with incorrect decisions in the code, or missing sections of functionality
- By testing with **at least one value** from every equivalence partition, where every value should be **processed in the same way**, equivalence partition testing attempts to find these faults

Fault at boundaries

- Equivalence partition tests are not designed to find faults at the values at each end of an equivalence partition
- If we inject a fault which moves the boundary value for the processing that returns DISCOUNT, then we do not expect to see any failed tests

```
33         if (bonusPoints>threshold)
34             rv=DISCOUNT;

33         if (bonusPoints>=threshold) // fault
34             rv=DISCOUNT;
```

Error Hiding

```
1 // return true if both x and y are valid
2 //      (within the range 0..100)
3 // otherwise return false to indicate an error
4 public static boolean valid(int x, int y) {
5     if (x<0 || x>100) return false;
6     if (y<0 || y>1000) return false;
7     return true;
8 }
```

- Fault on line 6 – should use 100, not 1000
- Example Error TCIs: x=500, y=500
- Test with multiple error TCIs
 - assertFalse(valid(500,500)) – PASSES
- Does NOT find the error on line 6 – line 5 returns false first
- The Error TCI “y=500” is NOT tested

9. Evaluation

- Testing with equivalence partitions provides a **minimum level** of black-box testing
- At least one value has been tested from every input and output partition...
- ...using a minimum number of tests cases
- Ensure correctness of basic data processing
- Do not exercise: different decisions made in the code
- Decisions frequent source of mistakes – generally reflect **boundaries** of input partitions, or **combinations** of inputs requiring particular processing
- These issues will be addressed in later techniques

Boundary Value Analysis



Testing with Boundary Value Analysis

- Boundary values are the minimum and maximum values for each Equivalence Partition
- Having identified the partitions, identifying the boundary values is straightforward
- The goal is to verify that the software works correctly at these boundaries

Typical Fault in Boundary Value Handling

- Line 3 contains a fault
!(x<100) is wrong
- Either !(x<=100) or (x>100) would be correct

```
1 // Return true is x is greater than 100
2 public void greater(int x) {
3     if (!(x<100)) return true;
4     else return false;
5 }
```

Example

- Continue testing OnlineSales.giveDiscount()
- Summary – the method returns:

FULLPRICE if bonusPoints≤120 and not a goldCustomer
 FULLPRICE if bonusPoints≤80 and a goldCustomer
 DISCOUNT if bonusPoints>120
 DISCOUNT if bonusPoints>80 and a goldCustomer
 ERROR if any inputs are invalid (bonusPoints<1)

Specification-Based Ranges

- bonusPoints

Long.MIN_VALUE	0	1	80	81	120	121	Long.MAX_VALUE
----------------	---	---	----	----	-----	-----	----------------

- goldCustomer

true	false
------	-------

Single-Valued Partitions

- Return Value

FULLPRICE	DISCOUNT	ERROR
-----------	----------	-------

Single-Valued Partitions

2. Test Coverage Items

TCI	Parameter	Boundary Value	Test Case
BV1*	bonusPoints	Long.MIN_VALUE	To be completed later
BV2*		0	
BV3		1	
BV4		80	
BV5		81	
BV6		120	
BV7		121	
BV8		Long.MAX_VALUE	
BV9	goldCustomer	true	
BV10		false	
BV11	Return Value	FULLPRICE	
BV12		DISCOUNT	
BV13		ERROR	

3. Test Cases (complete)

ID	TCI Covered	Inputs		Exp. Results
		bonusPoints	goldCustomer	
T2.1	BV3,9,11	1	true	FULLPRICE
T2.2	BV4,10,[11]	80	false	FULLPRICE
T2.3	BV5,[10,11]	81	false	FULLPRICE
T2.4	BV6,[10,11]	120	false	FULLPRICE
T2.5	BV7,[10],12	121	false	DISCOUNT
T2.6	BV8,[10,12]	Long.MAX_VALUE	false	DISCOUNT
T2.7	BV1*,13	Long.MIN_VALUE	false	ERROR
T2.8	BV2*,[13]	0	false	ERROR

Completed Test Coverage Item Table

TCI	Parameter	Boundary Value	Test Case
BV1*	bonusPoints	Long.MIN_VALUE	T2.7
BV2*		0	T2.8
BV3		1	T2.1
BV4		80	T2.2
BV5		81	T2.3
BV6		120	T2.4
BV7		121	T2.5
BV8		Long.MAX_VALUE	T2.6
BV9	goldCustomer	true	T2.1
BV10		false	T2.2
BV11	Return Value	FULLPRICE	T2.1
BV12		DISCOUNT	T2.5
BV13		ERROR	T2.7

Evaluation

- Boundary value analysis enhances the testing provided by equivalence partitions
- Experience indicates that this is likely to **find significantly more errors** than equivalence partitions
- Boundary value analysis tests two **additional** values from every input and output partition, the minimum and maximum
- These tests provide some assurance that the **correct decisions** are made in the code
- Boundary value analysis provides exactly the same test coverage items as equivalence partitions for boolean and for enumerated parameters

BVA: Decisions and Combinations

- Boundary value analysis does **not** explore the decisions in detail
- In particular BVA does not explore decisions associated with different combinations of inputs
- Combinations of inputs will be addressed next

Combinatorial testing



Motivation:

- verification of reactive systems
 - many configuration options
 - incomplete/unavailable specification
- go for BB testing, but....
 - partition testing not very effective in revealing faults
 - boundary testing does not exercise all combinations of inputs

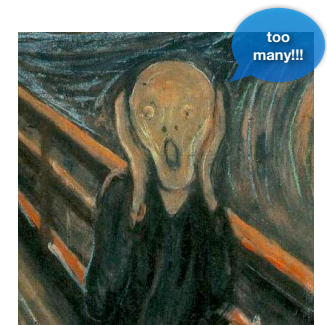
Combinatorial testing

- cover all combinations of inputs?
 - requires only knowledge of the input domain
 - still lots of tests

Example: 20 variables, 10 values each:

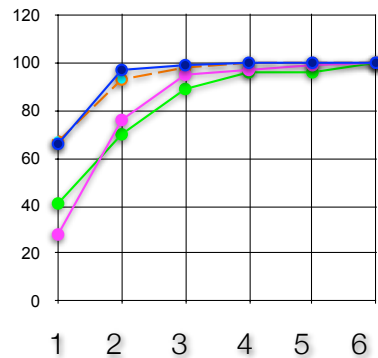
10^{20} combinations

A test for each atom in the universe



Feature Interaction testing

- all failures triggered by 1 to 6-way interaction [Kuhn, Okun@NIST]



Four application domains

- Medical
- Browser
- Server
- NASA

Feature Interaction testing

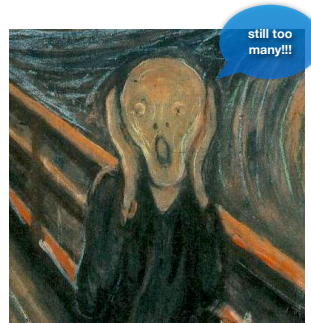
- Test combinations up to a given size (degree/strength)
- Most failures triggered by interaction of **just pairs** of input settings
 - test coverage items are all pairs of input parameter values
- empirical data show pairwise testing detect usually over 60% of faults
- go for higher interaction degrees, if required

k-wise Combinatorial Interaction test suites

For n variables with v values, k -way combinations $\binom{n}{k} v^k$

- Example:
- 4-way combinations of 30 parameters, 5 values each: **3,800** test goals

- Task: build a **covering array**
- minimal, if possible
- Task is NP-complete



COVERING ARRAY

- $CA_\lambda(N; t, K, V)$, $\lambda=1$
- $MCA_\lambda(N; t, K, V)$, $V=\{v_1, v_2, \dots, v_K\}$
 - $V=\{2, 2, 2, 2, 3, 3, 4\} \Rightarrow 2^4 3^2 4^1$
- $OA_\lambda(N; t, K, V)$
- latin squares



OLD CHINESE MAGIC SQUARE

Existing tools

- commercial tools: AETG, testCover, ...
- research tools: IPO - PairTest, IBM whitch, MS Pict, TTuples...
- www.pairwise.org

Most focus on optimization for size and speed

- $16 \cdot 10^{26} \rightarrow 256$ tests,
- $2^{120} \rightarrow 10$ tests

SPIN model checker

- $MCA(N; 2, 2^{1345})$, i.e., space of $8.3 \cdot 10^6$ different system configurations.
- 13 pairwise constraints
- relate 9 of the 18 factors
- no implied forbidden pairs.
- enforcing just 1 of the 13 constraints eliminates over 2 million configurations.
- **100% Pairwise coverage in 19 tests only**

SPIN verifier

- $MCA(N; 2, 2^{4232411})$;
- space of $1.7 \cdot 10^{20}$ different configurations.
- 49 constraints
- 47 over pairs of combinations and 2 over triples
- 33 of the 55 factors are involved in constraints.
- 9 implied forbidden pairs
- a single constraint eliminates more than $2 \cdot 10^{19}$

- **Pairwise coverage in 35 tests**

GCC 4.1

- $MCA(N; 2, 2^{189} 3^{10})$
- state space of $4.6 \cdot 10^{61}$ different configurations.
- 40 constraints, 3 three-way and 37 pairwise
- related to 35 of the 199 factors
- 2 implied forbidden pairs
- $1.2 \cdot 10^{61}$ conf. x constraint

- **Pairwise coverage in 20 tests**

RESULTS

	mAETG.SAT		SA.SAT	
	without constraints	with constraints	without constraints	with constraints
$t = 2$				
SPIN	33	41	27	35
Verifier	9.8 sec	32.2 sec	19.6 sec	31,595.2 sec
SPIN	25	24	16	19
simulator	0.4 sec	1.5 sec	25.6 sec	694.3 sec
GCC	24	23	16	20
	323.2 sec	371.6 sec	4,137.0 sec	18,186.2 sec
$t = 3$				
SPIN	100	108	78	95
simulator	6.3 sec	11.9 sec	1,577.5 sec	13,337.4 sec

Table 3: CIT size and time for GCC and SPIN

Statement Coverage



Testing with Statement Coverage

- Statement coverage (SC) testing (or more formally testing with statement coverage criteria) ensures that **every line, or statement**, of the source code is **executed** during tests, while verifying that the **output is correct**

Definition:

a statement is a line in the source code that can be executed
(**Not all lines can be executed**: comments, blank lines, {}'s, etc.)

Test Coverage Items

- Each statement in the source code is a test coverage item
 - Every **executable** statement
- Normally, just consider 'extra' TCIs (not already covered by BB tests)
- Normally, a single line of source is regarded as being a statement
- Issues:
 - Multiple statements on one line
 - Multi-line statements

Identifying Unexecuted Statements

- Using coverage results from previous tests (black-box tests), unexecuted statements easily identified
 - You can use “JaCoCo” – see www.jacoco.org
- For complex code, a **Control-Flow Graph (CFG)** may be developed first to help with understanding the code flow at a more abstract level, but these are seldom required for statement coverage
- Statement coverage tests may be developed before black-box tests, though this is not usual practice
 - In this case, CFGs are traditionally used to assist in developing the tests

```

19. public static Status giveDiscount(long bonusPoints, boolean goldCustomer)
20. {
21.     Status rv = FULLPRICE;
22.     long threshold=120;
23.     if (bonusPoints<0)
24.         rv = ERROR;
25.     else if (goldCustomer)
26.     {
27.         threshold = 80;
28.         if (bonusPoints>threshold)
29.             rv=DISCOUNT;
30.     }
31.     if (bonusPoints==43) // fault 4
32.         rv=DISCOUNT;
33.     return rv;
34. }
    
```



Coverage results

- Examine the source code coverage report
- determine which lines have not been executed
- Uncovered item at line 38
- identify the condition that will cause line 38 to execute: bonusPoints=43

```

19. * FULLPRICE - charge the full price<br>
20. * ERROR - invalid inputs
21. */
22. public static Status giveDiscount(long bonusPoints
23. {
24.     Status rv = FULLPRICE;
25.     long threshold=120;
26.
27.     if (bonusPoints<=0)
28.         rv = ERROR;
29.
30.     else {
31.         if (goldCustomer)
32.             threshold = 80;
33.         if (bonusPoints>threshold)
34.             rv=DISCOUNT;
35.     }
36.
37.     if (bonusPoints==43) // fault 4
38.         rv = DISCOUNT;
39.
40.     return rv;
41. }
42.
43. }
    
```

Adding Extra Test Cases

- Unlike in black-box testing, there is no differentiation between error and non-error cases for white-box testing

ID	TCI	Inputs		Exp. Results
		bonusPoints	goldCustomer	
T4.1	SC1	43	false	FULLPRICE

Strengths

- Provides a minimum level of coverage by executing all the statements in the code at least once
- There is a significant risk in releasing software before every statement has been executed at least once during testing
 - its behaviour has not been verified, and may well be faulty
- Statement coverage can generally be achieved using only a small number of extra tests

The **full test** implementation **includes** the previously developed equivalence **partition and boundary** value analysis tests!!

Weaknesses

- Can be difficult to determine the required input parameter values
- Hard to test code only executed in unusual circumstances
- **Does not provide coverage for the NULL else**
 - `if (number < 3) number++;`
 - Statement coverage does not force a test case for `number ≥ 3`
- Not demanding of compound decisions
 - `if ( (a>1) || (b==0) ) then x = x/a;`
 - No test cases for the different boolean conditions that may cause the decision to be true or false
 - No test cases for the possible combinations of the boolean conditions

Branch Coverage



Branch Coverage

- Essentially the same, except that we **consider branches instead of statements**
- Note that there are different ways to measure branches:
 - Every statement with a whole decision being true or false
 - Every binary condition in a decision being true or false
- JaCoCo measures the outcome of every Boolean condition in a decision as a separate branch
 - Based on the Java bytecode

Branches Explained

```
1 if (x && y)
2     z = 10;
3 else
4     z = 20;
```

• Decisions-Based

Two branches:

1. Line 1 to line 2 if x and y are true
2. Line 1 to line 4 otherwise

• Conditions-Based

Four branches:

1. Condition x to y if x is true
2. Condition x to line 4 if x is false
3. Condition y to line 2 if y is true
4. Condition y to line 4 if y is false

Fault Model

- Branches that has not been taken in previous tests (**untaken branches**) **may contain a fault**
- As for statement coverage, these tend to be associated with edge cases, or other unusual circumstances
- Branch coverage tests with input values carefully selected to **ensure that every branch is taken** during test execution
- These tests attempt to find **faults associated with individual branches** in the source code

Using JaCoCo

- Can use previous coverage to start from
- JaCoCo counts the outcome of each **boolean condition** as a branch
- a different tool may count the outcomes of each **decision** as branches instead
- This would reduce the number of branches, which leads to slightly reduced test effectiveness

```
27. if (bonusPoints<=0)
28.     rv = ERROR;
29.
30. else {
31.     if (goldCustomer && bonusPoints!=93) // fa
32.         threshold = 80;
33.     1 of 4 branches missed. (nts>threshold)
34.     rv=DISCOUNT;
35. }
36.
37. return rv;
38. }
39.
```

Strengths and Weaknesses

- Strengths:
 - Branch coverage is a **stronger** form of testing than statement coverage: 100% branch coverage guarantees 100% statement coverage – but the test data is harder to generate
 - **Resolves the NULL else problem**
- Weaknesses:
 - Can be **difficult to determine the required input** parameter values
 - If the tool only counts decisions as branches, or if a control flow graph has been manually developed, then it is undemanding of compound decisions. In these cases it does not explore all the different reasons (i.e. the boolean conditions) for the decision evaluating as true or false

All Paths Coverage



Introduction to All Paths Coverage (AP)

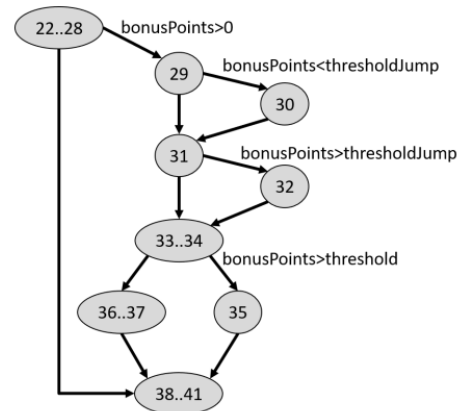
- **The strongest form** of white-box testing based on “**program structure**”
- If you achieve AP, then you have also achieved SC and BC
- All the paths, from entry to exit of a block of code (usually a method; the test item), are executed during testing
- Developing all paths tests is complex and time consuming
- In practice it is seldom used (only considered for critical software)

CFGs

- A control flow graph (CFG) is a directed graph showing the flow of control through a segment of code
- These are mainly used to represent code at the source code level
 - Each **node** in the graph represents one or more indivisible statements
 - Each **edge** represents a jump or branch in the flow of control
- A node with two exits represents a block of code terminating in a decision, with true and false exits (such as an if statement)
- The CFG is language independent – a simplified model of the code
- This allows sequential blocks of code, branches, and paths, to be easily identified

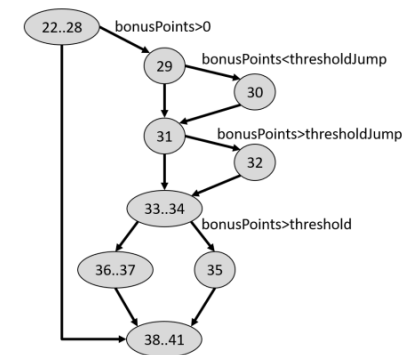
Control-Flow graph

- Definition: an **end-to-end path** is a single flow of execution from the start to the end of a section of code
- This path may include multiple executions of any loops on the path
- **Complessità cicломatica**
 - $cc = \text{numero di archi} - \text{numero di nodi} + 2$
- E' il **numero minimo** di test (percorsi) da effettuare per testare tutte le condizioni (**coprire tutto il cfg**)
- Non conta tutti i possibili **percorsi distinti**



Esempio

```
22 public static Status giveDiscount(long bonusPoints, boolean
    goldCustomer)
23 {
24     Status rv = ERROR;
25     long threshold=goldCustomer?80:120;
26     long thresholdJump=goldCustomer?20:30;
27
28     if (bonusPoints>0) {
29         if (bonusPoints<thresholdJump)
30             bonusPoints -= threshold;
31         if (bonusPoints>thresholdJump)
32             bonusPoints -= threshold;
33         bonusPoints += 4*(thresholdJump);
34         if (bonusPoints>threshold)
35             rv = DISCOUNT;
36         else
37             rv = FULLPRICE;
38     }
39
40     return rv;
41 }
```



Standard CFGs

```
1 int f()
2 {
3   int x,y;
4   x = 10;
5   y = x+3;
6   return y;
7 }
```

1..7

Figure 7.11: CFG for Sequence

```
1 int f(int a)
2 {
3   int x=0;
4   if (a>10)
5     x=a;
6   return x;
7 }
```

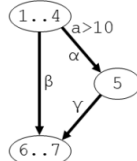


Figure 7.12: CFG for Selection (if-then)

```
1 int f(int a)
2 {
3   int x=0;
4   if (a>10)
5     x=a;
6   else
7     x=-a;
8   return x;
9 }
```

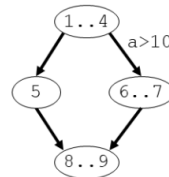


Figure 7.13: CFG for Selection (if-then-else)

```
...
1 switch (a) {
2   case 0:
3     b=33;
4     break;
5   case 1:
6     b=-44;
7     break;
8   default:
9     ok=false;
10    break;
11 }
...
```

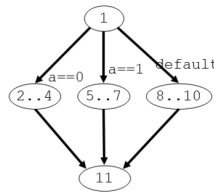


Figure 7.14: CFG for Selection (switch)

More Standard CFGs

```
1 int f(int a)
2 {
3   int x=a;
4   while (x>10)
5     x=x/2;
6   return x;
7 }
```

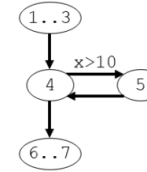


Figure 7.15: CFG for Iteration (while)

```
1 int f(int a)
2 {
3   int x=a;
4   do {
5     x=x/2;
6   } while (x>10);
7   return x;
8 }
```

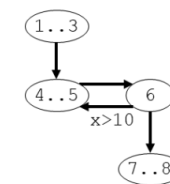


Figure 7.16: CFG for Iteration (do-while)

```
1 int f(int a)
2 {
3   int x=0;
4   for (int i=0; i<a; i++)
5     x = x+a+i;
6   return x;
7 }
```

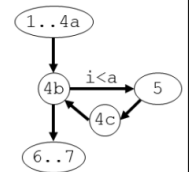


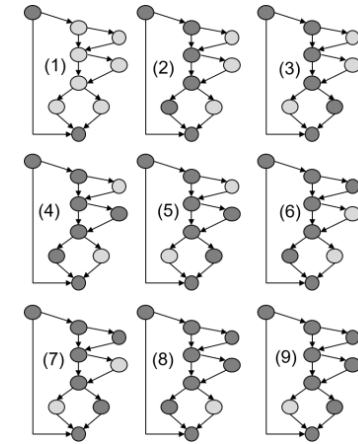
Figure 7.17: CFG for Iteration (for)

Control Flow Graphs (CFGs)

- In statement and branch testing:
 - Control Flow Graphs (CFGs) are seldom used in practice
 - Tests can be more efficiently developed to supplement black-box tests
 - Using automated tools to measure the coverage
- However, in **all paths testing**, **CFGs are essential**

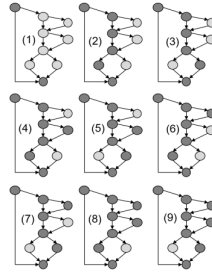
Identifying the Candidate Paths

- By tracing all the start-to-end paths through the CFG, the candidate paths can be identified
- These are candidate paths: not of them are logically possible to take
- A systematic approach must be taken to avoid missing paths
- Work through the CFG starting at the top, and taking the left hand branches first



Identifying the Possible Paths

- Path 1, nodes (22..28)–(38..41) covered by bonusPoints=-100, and goldCustomer=false (T1.4)
- Path 2, nodes (22..28)–(29)–(31)–(33..34)–(36..37)–(38..41) **not possible**
- Path 3, nodes (22..28)–(29)–(31)–(33..34)–(35)–(38..41) covered by 20, true or 30, false
- Path 4, nodes (22..28)–(29)–(31)–(32)–(33..34)–(36..37)–(38..41) covered by 1, true (T2.1)
- Path 5, nodes (22..28)–(29)–(31)–(32)–(33..34)–(35)–(38..41) covered by 40, true (T1.1)
- Path 6, nodes (22..28)–(29)–(30)–(31)–(33..34)–(36..37)–(38..41) **not possible**
- Path 7, nodes (22..28)–(29)–(30)–(31)–(33..34)–(35)–(38..41) **not possible**
- Path 8, nodes (22..28)–(29)–(30)–(31)–(32)–(33..34)–(36..37)–(38..41) **not possible**
- Path 9, nodes (22..28)–(29)–(30)–(31)–(32)–(33..34)–(35)–(38..41) **not possible**



Analysis Results

- This analysis of the CFG has produced three results:
 - The possible end-to-end paths
 - Where available, existing test data that causes a path to be taken
 - Where no previous test causes a path to be taken, criteria or values for the new test data

Path	Input Values	Existing Test
1	(-100,false)	T1.4
3	(20,true) or (30,false)	New test required
4	(1,true)	T2.1
5	(40,true)	T1.1

Fault Model

- The all-paths fault model is where a particular sequence of operations, reflected by a particular path through the code, does not produce the correct result
- These faults are often associated with complex or deeply nested code structures where the **wrong functionality** is executed for a specific situation
- Trovo errori funzionali complessi che si manifestano solo con l'esecuzione di **una sequenza più lunga di un singolo branch**

Strengths and Weaknesses

- Strengths**
 - Covers all possible paths, which may have not been exercised using other methods
 - Guarantees statement coverage and branch coverage
- Weaknesses**
 - Difficult and time consuming
 - All-paths does not explicitly evaluate the boolean conditions in each decision
 - Does not explore faults related to incorrect data processing (e.g. bitwise manipulation or arithmetic errors)
 - Does not explore non-code faults (for example, faults in a lookup table)

WBT: Some More Techniques

- Dataflow coverage/Definition-Use pairs
- Condition Coverage (CC)
- Decision Coverage (DC)
- Decision/Condition Coverage (DCC)
- Multiple Condition Coverage (MCC)
- Modified Condition/Decision Coverage (MCDC)
- Test ranking

Dataflow coverage/Definition-Use pairs

- Each **path between** the writing of a value to a variable (**definition**) and its subsequent reading (**use**) is executed during testing
 - Definition: $x=10$
 - Use: $y=x+20$ or if ($x>100$)
- Every possible du pair is a test coverage item
- Strengths
 - This is a strong form of testing.
 - It generates test data in the pattern that data is manipulated in the program
- Weaknesses
 - The number of test cases can be very large
 - Hard to handle object references (C/C++ pointers) and arrays

Condition Coverage (CC)

- Each **boolean condition** within a decision is tested for its true and false values
- TCI for “if (a || (b && c))”
 - a and !a
 - b and !b
 - c and !c
- Does not ensure decision is true and false
- Can be difficult to determine the required input parameter values

Decision Coverage (DC)

- In decision coverage (often abbreviated to DC), **every decision** is evaluated to true and false
- This is **equivalent to branch coverage** if a coverage measurement tool is used that identifies decisions as branches (or if a control flow graph has been used)

Decision/Condition Coverage (DCC)

- In decision condition coverage (often abbreviated to DCC) , tests are generated that **both** cause **every decision** to be taken at least once (decision coverage), **and** also **every** boolean **condition** to be true and false at least once (condition coverage)
- TCIs
 - Each value of each conditions (true and false)
 - Each value of each decision (true and false)
- Doesn't test every combination of conditions
- Can be difficult to determine required input parameter values

Multiple Condition Coverage (MCC)

- Tests are generated to cause **every possible combination** of boolean **conditions** for **every decision** to be tested
- Use a Decision Table
- Each decision with **n independent** boolean conditions has **2ⁿ** test coverage items
- A large number of TCIs
- Can be difficult to determine required input parameter values

Modified Condition/Decision Coverage (MCDC)

- MCC generates a very large number of tests
- Reduced by **only considering those combinations** of Boolean conditions that cause a discernible **effect on the output**
- The test conditions:
 - decision condition coverage, plus
 - additional conditions to verify the independent effect of each boolean condition on the output
- Each decision has two test coverage items (true and false)
- Every boolean condition in each decision has two test coverage items
- In addition, test coverage items must be created that show the effect on the output of changing each of the boolean conditions independently

MCDC Example

```
1  int func(int a, int b) {
2      int x=100;
3      if ( (a>10) || (b==0) ) then x = x/a;
4      return x;
5  }
```

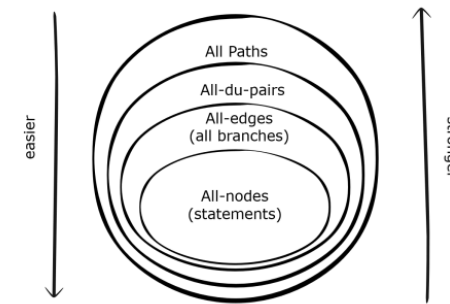
- Tests must ensure each condition true and false:
 - a>10
 - b==0
- Tests must ensure each decision true and false:
 - (a>10) || (b==0)
- Tests must ensure the effect on the output value of changing the value of every boolean condition is shown

MCDC Example/Test Data

```
1  int func(int a, int b) {  
2      int x=100;  
3      if ( (a>10) || (b==0) ) then x = x/a;  
4      return x;  
5  }
```

- Inputs (a=50,b=1) should result in the output value: 2
- (a=5,b=1) should result in the output value: 100
 - This shows the independent effect of changing the boolean condition (a>10)
- The input data (a=5,b=0) should result in the output value: 20
 - This shows the independent effect of changing the boolean condition (b==0)
 - Non preudo (a=50, b=0) perché risulterebbe sempre 2 come (a=50, b=1)

Structural Test ranking



15 min. break



Modeling and Design



Dealing with Complexity

Object oriented design give us these tools:

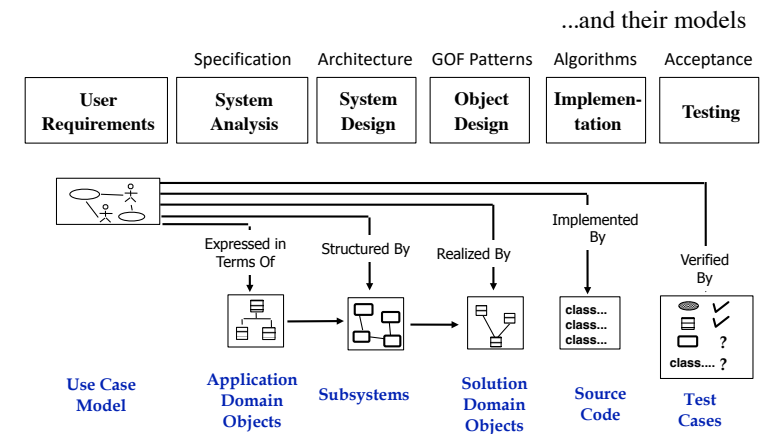
1. Abstraction
2. Decomposition
3. Hierarchy

Models are used to provide abstractions

• System Models:

- **Functional model:** What are the functions of the system? How is data flowing through the system? ==>(use case diagrams)
- **Object Model:** What is the structure of the system? What are the objects and how are they related? ==>(class diagrams)
- **Dynamic model:** How does the system react to external events? How is the event flow in the system ? ==>(sequence/activity/state diagrams)

Summary: Software Lifecycle Activities



First two Steps: deal with requirements

– Requirements Elicitation:

- Definition of the system in terms understood by the customer (“**Problem statement**”) leads to a “**Requirements Document**” document
- Il vocabolario è quello dell’utente (del dominio)

– Requirements Analysis:

- Technical specification of the system in terms understood by the developer: “**System (requirements) specification (Model)**”
- Il vocabolario è quello dello sviluppatore (tecnico)

Requirements Elicitation

- Very challenging activity
- Requires collaboration of people with different backgrounds
 - Users with application domain knowledge
 - Developer with solution domain knowledge (design knowledge, implementation knowledge)
- Bridging the gap between user and developer:
 - *Scenarios*: Example of the use of the system in terms of a series of interactions with between the user and the system
 - *Use cases*: Abstraction that describes a class of scenarios

Ingredients of a system specification

- Current situation: The Problem to be solved
- Description of one or more scenarios
- Requirements
 - Functional and Nonfunctional requirements
 - Constraints (“pseudo requirements”)
- Project Schedule
 - Major milestones that involve interaction with the client including deadline for delivery of the system
- Target environment
 - The environment in which the delivered system has to perform a specified set of system tests
- Client Acceptance Criteria
 - Criteria for the system tests

What is usually not in the requirements?

- System architecture
- Development methodology
- Development environment
- Implementation language
- Reusability

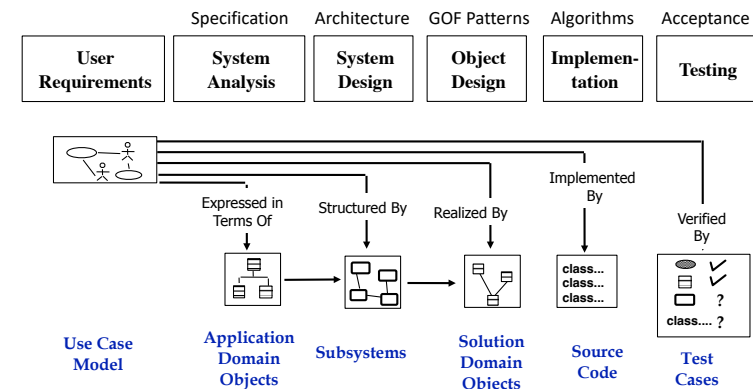
Requirements Analysis: Object Modeling

How to identify objects from requirements



Summary: Software Lifecycle Activities

...and their models



Finding Participating Objects in Use Cases

- Pick a *use case* and look at its *flow of events*
 - Look for recurring nouns
 - Identify real world entities NAMES that the system needs to keep track of
 - Identify real world PROCEDURES that the system needs to keep track of
 - Identify DATA sources or sinks (e.g., Printer)
 - Identify interface ARTIFACTS (e.g., PoliceStation)
- Expect that some objects are still missing and need to be found:
 - Model the flow of events (in user stories/use cases) with **activity diagrams**

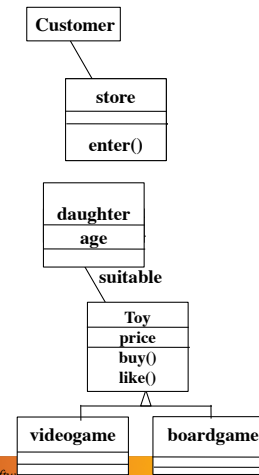
Example: Flow of events

- The customer enters a store with the intention of buying a toy for his child with the age of n.
- Help must be available within less than one minute.
- The store owner gives advice to the customer. The advice depends on the age range of the child and the attributes of the toy.
- The customer selects a dangerous toy which is kind of unsuitable for the child.
- The store owner recommends a more suitable doll.

Mapping parts of speech to object model components [Abbott, 1983]

Part of speech	Model component	Example
Proper noun	object	Jim Smith
Improper noun	class	Toy, doll
Doing verb	method	Buy, recommend
being verb	inheritance	is-a (kind-of)
having verb	aggregation	has an
modal verb	constraint	must be
adjective	attribute	3 years old
transitive verb	method	use
intransitive verb	method (event)	enter, depends on

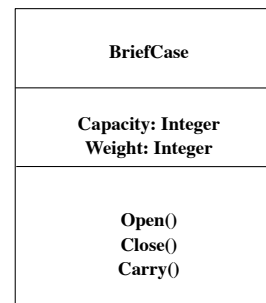
Generation of a class diagram from flow of events



- The **customer** enters the **store** to buy a toy. It has to be a **toy** that his **daughter** likes and it must cost **less than 50 Euro**. He tries a **videogame**, which uses a data glove and a head-mounted display. He likes it.

An assistant helps him. The suitability of the game **depends on** the **age** of the child. His daughter is only 3 years old. The assistant recommends another **type of toy**, namely a **boardgame**. The customer buy the game and leaves the store

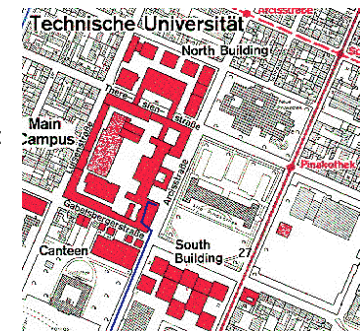
Class identification: Modeling a Briefcase



- Supponiamo di avere identificato questo oggetto nel mio modello...

What is modeling?

- Modeling consists of building an abstraction of reality.
- Abstractions are **simplifications** because:
 - They ignore irrelevant details and
 - They only represent the relevant details
- What is relevant or irrelevant **depends on the purpose** of the model.



A new Use for a Briefcase: Sit On It

Why did we not model it as a chair?



BriefCase
Capacity: Integer Weight: Integer
Open() Close() Carry() SitOnIt()

- Dal modello funzionale deriva SitOnIt(), ed è l'unica applicabile a quell'oggetto...

Correct object identification

Why did we model the thing as "Briefcase"?

What do we do if the SitOnIt() operation is the most frequently used operation?

The briefcase is only used for sitting on it. It is never opened nor closed.
Is it a "Chair" or a "Briefcase"?

E' **in base all'uso, alla loro interfaccia**, che il tipo degli oggetti del dominio è identificabile correttamente

Partire dalla loro descrizione non funzionale, ovvero per attributi, potrebbe essere fuorviante

la modellazione funzionale dei requisiti è fondamentale anche per validare la sua modellazione ad oggetti (entità)

Analysis: Dynamic Modeling



Dynamic Modeling

- Definition of dynamic model:
 - A collection of multiple state chart diagrams, one state chart diagram for each class with important dynamic behavior.
- Purpose:
 - Detect and supply **methods** for the object model
- How do we do this?
 - Start with use case or scenario => activity diagram
 - Model interaction between objects => sequence/interaction diagram
 - Model dynamic behavior of a single object => statechart diagram

2. Decomposition

- A technique used to master complexity ("divide and conquer")
- **Functional decomposition**
 - The system is decomposed into modules
 - Each module is a **major processing step (function)** in the application domain
 - Modules can be decomposed into smaller modules
- **Object-oriented decomposition**
 - The system is decomposed into classes ("objects")
 - Each class is a **major abstraction** in the application domain
 - Classes can be decomposed into smaller classes

Which decomposition is the right one?

Start with a description of the functionality (Use case model). Then proceed by finding objects (object model).

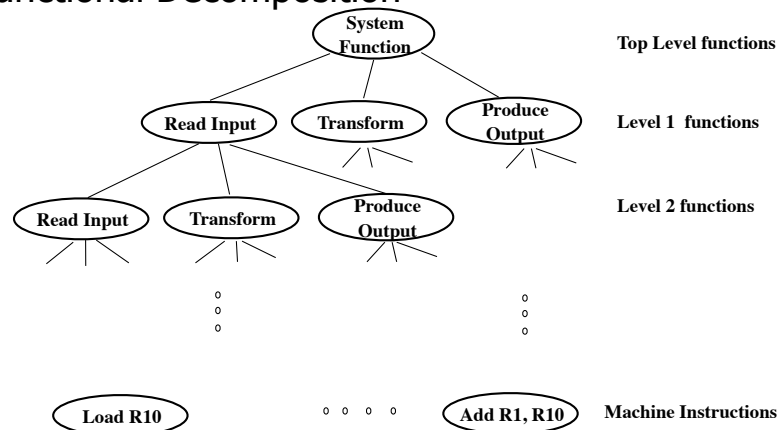
If you think you are politically correct, you probably want to answer: Object-oriented. But that is actually wrong.

Both views are important

Functional decomposition **emphasises** the ordering (of use) of **operations**

Object-oriented decomposition **emphasizes the agents that cause the operations**

Functional Decomposition



Functional decomposition

- Decompongo il goal principale in sottoproblemi (funzioni) più semplici
 - in uno stesso livello di astrazione, trovo altre funzionalità (sottoproblemi) di altri user goals di livello superiore, trovo anche ripetizioni
 - È un processo di decomposizione top-down, classico prima dell'avvento della paradigma object oriented
- Functionality is spread all over the system
 - All functions will have a common (algorithmic) structure:
get input - transform - put output
 - La realizzazione di una funzionalità (goal e suoi sotto-goal) è **spalmata orizzontalmente e verticalmente** su tutti i livelli sottostanti di decomposizione
 - Non c'è alcuna traccia di modularità in questa decomposizione...

Functional Decomposition

- ♦ Maintainer must understand the whole system to make a single change to the system
- ♦ Consequence:
 - ♦ Codes are hard to understand
 - ♦ Code that is complex and impossible to maintain
 - ♦ User interface is often awkward and non-intuitive

Devo fermarmi ad un livello di granularità corretto, non decomporre all'infinito

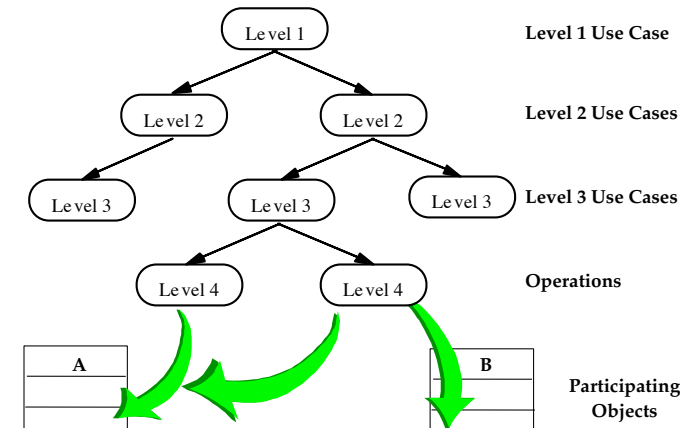
La granularità corretta è quella degli oggetti del dominio: che devo avere già identificato

From Use Cases to Objects

Decompongo funzionalmente in operazioni sempre più semplici

Mi fermo quando le funzionalità utente sono ad un livello di astrazione che corrisponde a responsabilità associabili agli oggetti che ho identificato nel sistema

Possono ancora esserci possibili duplicazioni o scelte non chiare...

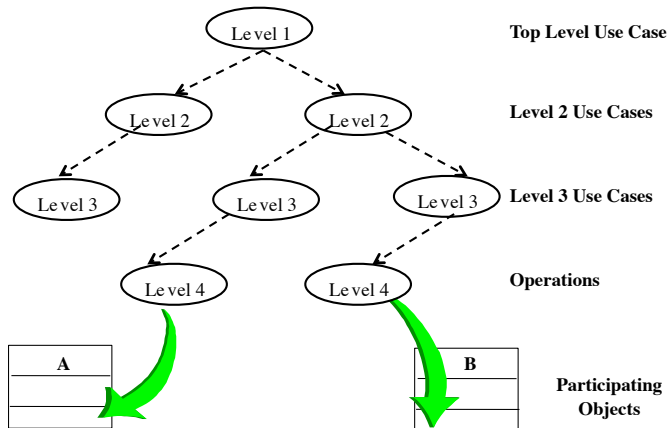


From Use Cases to Objects

Non mappare i gruppi di funzionalità individuati direttamente e rigidamente in corrispondenti classi, one-to-one.

Identifica le classi indipendentemente dalle funzionalità e poi mappa le funzionalità una per una dentro le classi più appropriate

Perché la suddivisione in gruppi delle funzionalità presenti nel sistema è utile ma non coincide con la suddivisione in categorie di entità presenti nel sistema.



Summary: Order of activities in modeling

1. Formulate a few scenarios with help from the end user and/or application domain expert.
2. Extract the use cases from the scenarios, with the help of application domain expert.
3. Analyse the flow of events, for example with Abbot's textual analysis.
4. Generate the class diagrams, which includes the following steps:
 1. **Class identification (textual analysis, domain experts).**
 2. **Identification of attributes and operations (sometimes before the classes are found!)**
 3. **Identification of associations between classes**
 4. **Identification of multiplicities**
 5. **Identification of roles**
 6. **Identification of constraints**

Relations in models

We got abstractions and decomposition

This leads us to **chunks** (classes, objects)

Another way to deal with complexity is to **provide simple relationships** between the chunks

One of the most important relationships is hierarchy (but not the only one)

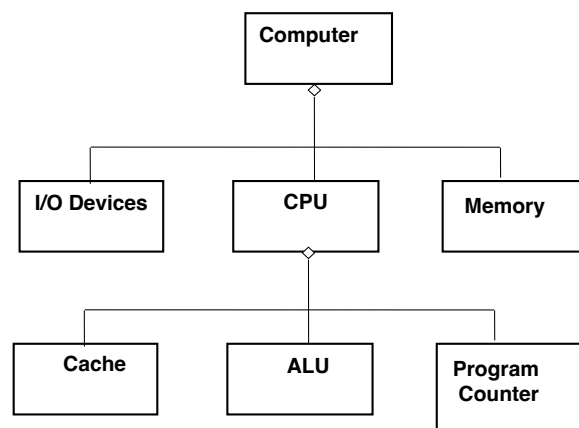
This lead us to talk about architectures....

3. Hierarchy

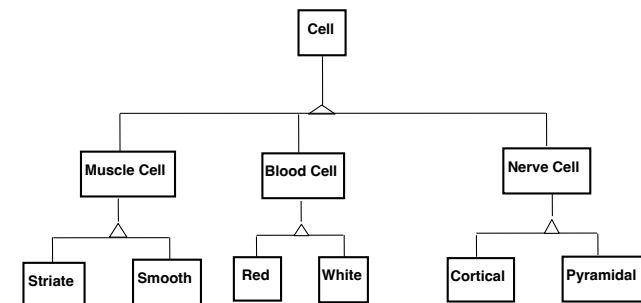
- ♦ 3 important hierarchies
 - ♦ "Part-of" hierarchy (UML composite structure diagrams)
 - ♦ "Is-kind-of" hierarchy (is-a only class diagrams: taxonomy)
 - ♦ "Uses" hierarchy (dependences/uses only class diagrams)

Part of Hierarchy

UML Composite structure diagrams



Is-Kind-of Hierarchy (Taxonomy)

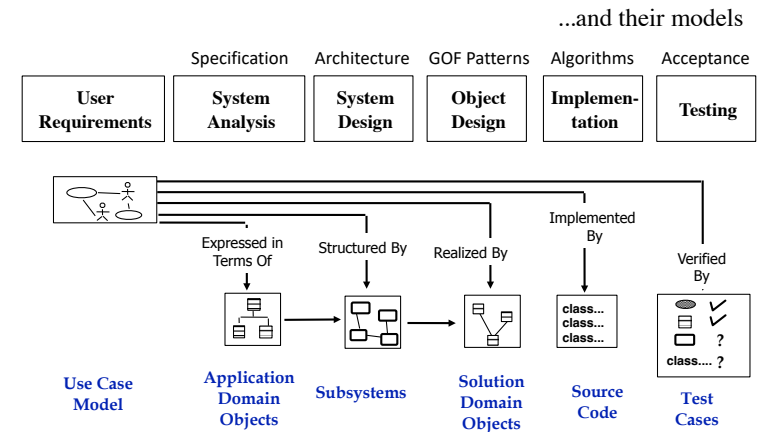


Can create separate Class diagrams to report the is-a hierarchy only

System Design



Summary: Software Lifecycle Activities



System Design

The design activity produces the software architecture (or software design)

The architecture of a software system defines the system in terms of computational components and interactions among those components. (Garlan&Shaw1996)

The task is to define the components and their relations:
That's why we used abstraction techniques in order to define MODULES and HIERARCHIES

Design principles

- How to select modules?
- How to define module interfaces?
- How to define USE relations?

Select Modules by grouping related functionalities, at high abstraction level
Select Modules by grouping related objects, at Mid-Low abstraction levels

How to select modules

- A module is a self contained unit
- USE interconnections with other modules should be minimized
- PRINCIPLE:
 - maximize cohesion and minimize coupling

How to select modules&interfaces

- Distinguish between what a module does for others and how it does that (its secrets)
- Minimize flow information to clients to maximize modifiability
- The interface is a contract with clients and must be stable
- GOLDEN PRINCIPLE: information hiding (Parnas 1974)
 - define what you wish to hide and design a module around it

How to select modules

Something "larger" than a class is needed to help organize large applications: packages (higher level modules)

- Granularità dei moduli è importante
 1. What are the best partitioning criteria?
 2. What are the relationships that exist between packages, and what design principles govern their use?
 3. Should packages be designed before classes (Top down)? Or should classes be designed before packages (Bottom up)?

The Reuse/Release Equivalence Principle (REP)

THE GRANULE OF REUSE IS THE GRANULE OF RELEASE.

- ONLY COMPONENTS THAT ARE RELEASED THROUGH A TRACKING SYSTEM CAN BE EFFECTIVELY REUSED.
- THIS GRANULE IS THE PACKAGE.

The Common Reuse Principle (CRP)

THE CLASSES IN A PACKAGE ARE REUSED TOGETHER.

IF YOU REUSE ONE OF THE CLASSES IN A PACKAGE, YOU REUSE THEM ALL.

So, **classes that tend to be (re)used together belong in the same package.**
i.e. a container and its iterators

The Common Closure Principle (CCP)

THE CLASSES IN A PACKAGE SHOULD BE CLOSED TOGETHER AGAINST THE SAME KINDS OF CHANGES.

A CHANGE THAT AFFECTS A PACKAGE AFFECTS ALL THE CLASSES IN THAT PACKAGE

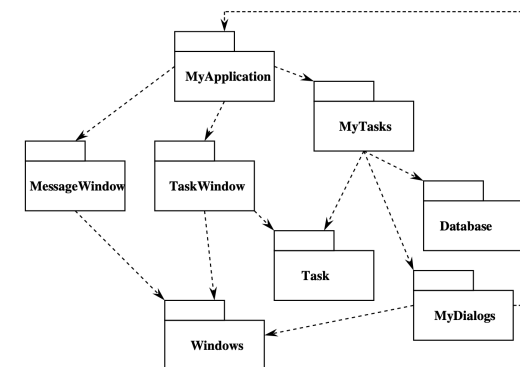
Gather together in **one place** all the classes that are **likely to change** for the **same reasons**.
So that a change in requirements has a good chance of affecting a minimal number of packages.

The Acyclic Dependencies Principle (ADP)

THE DEPENDENCY STRUCTURE BETWEEN PACKAGES MUST BE A DIRECTED ACYCLIC GRAPH (DAG).

THAT IS, THERE MUST BE NO CYCLES IN THE DEPENDENCY STRUCTURE.

Package diagram with cycles

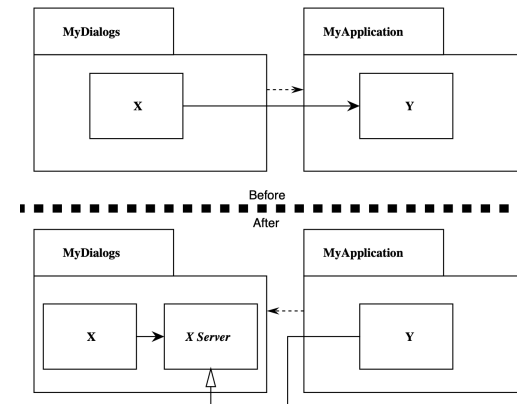


Per **testare MyDialog** gli devo linkare tutto il sistema: devo **fare una complete build** !

Breaking the cycle

1. Apply the Dependency Inversion Principle (DIP).
2. Create a new package that both MyDialogs and MyApplication depend upon. Move the class(es) that they both depend upon into that new package.

Dependency Inversion Principle (DIP)

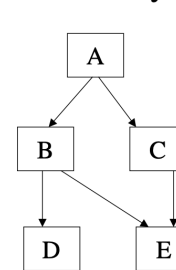


Design styles

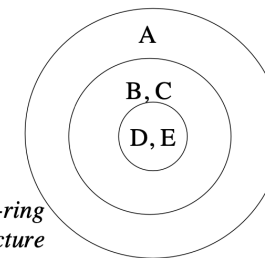
- Shared understanding of common design forms is typical of mature engineering fields
- Shared vocabulary of design idioms is codified in engineering handbooks
- Software is going in this direction
 - but there is less maturity

Layered system

- The system is organized through abstraction levels, as a hierarchy of abstract machines
- Hierarchy is given by the USE relation



the onion-ring structure



known uses

- Virtual Machines (Java)
- API
- Information systems (OSI, Internet)

Varriante: relaxed layered, posso usare i servizi di tutti i livelli sottostanti.

Pipes&filters



This is a pipeline

- Various control regimes are possible

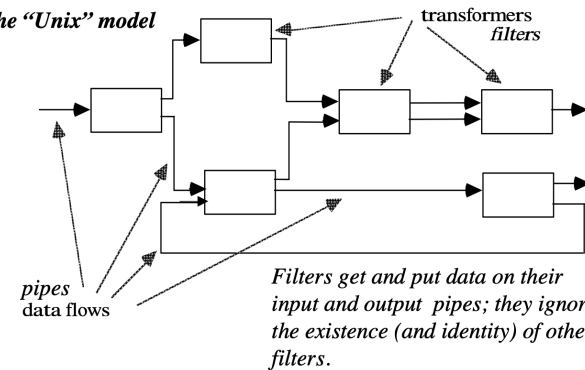
Batch: processare a “lotti”, cioè un lungo elenco di input tipicamente dato in un file.

Sequenziale: in modo sequenziale processo un input attivando un solo blocco per volta

Concorrente: tutti i blocchi sono attivi contemporaneamente, non aspetto di aver concluso tutti i passi per iniziare a processare l’input successivo.

Pipes&filter style

The “Unix” model



I componenti (filters) non condividono dati tra loro

I connettori (pipes) si limitano a trasportare dati da un canale di output di un componente a un canale di input dell’altro

Esempio: i programmi a carattere unix %>. ls -l | grep pippo

Pipes&filters

- + compositional

- overall behavior as composition of individual behaviors

- + reuse oriented

- any two filters can be put together in principle

- + modifications are easy

- can add/replace filters

- no persistency

- replications (*)

- tendency to batch organization

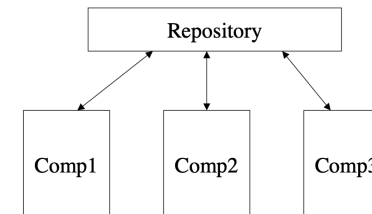
(*) Se una operazione va fatta più volte, l’unica opzione possibile è replicare il filtro corrispondente allungando la pipeline

Repository-based systems

- Components communicate only through a repository

Repository è dati: è uno storage, anche remoto

Modello **database**: è un repository passivo. Ha un componente aggiuntivo (dBMS) che sa gestire le transazioni garantendo la coerenza interna

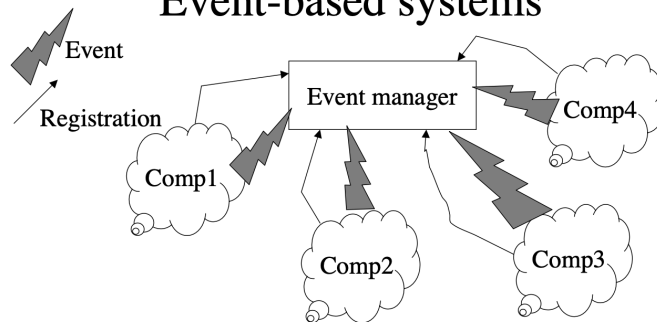


Modello **blackboard**: è un repository attivo. Alle modifiche allo stato del repository (causate dalle scritture) possono corrispondere attivazioni di altri componenti (letture-scritture).

Blackboard:

Known Uses => **No deterministic solution** strategy known. Several subsystems assemble their knowledge to build a possibly partial or approximate solution.

Event-based systems



- Events are broadcasted to all registered components
- No explicit naming of target component

Event-based systems

Facilita le strategie **d'integrazione**

i componenti sono tra loro privi di dipendenze dirette

Posso rimuovere o aggiungere componenti facilmente

Può avere problemi di **scalabilità**

i componenti sono **asincroni** tra loro: l'ordine degli eventi non è garantito

Perché il manager li possa rimbalzare nell'ordine di generazione, gli originatori dovrebbero marcarli e avere gli orologi perfettamente sincronizzati tra loro: non è una cosa banale.

Where are we right now?

- Three ways to deal with complexity:
 - Abstraction
 - Decomposition (Technique: Divide and conquer)
 - Hierarchy (Technique: Layering)
- Two ways to deal with decomposition:
 - Object-orientation and functional decomposition
 - Functional decomposition leads to unmaintainable code
 - Depending on the purpose of the system, different objects can be found
- What is the right way?
 - Start with a description of the functionality (Use case model). Then proceed by finding objects (object model).
- What modules and relations are needed?
 - Choose an architectural style and apply modular composition/decomposition following the reviewed design principles

Development Methods



Development Methods Help Manage Risks

- We use term risk informally for a potential challenge
 - Risk can be thought of as the cost of recovering if something goes wrong
- Process models tend to focus on specific risks, as we'll see
 - Agile methods are iterative, so they manage risk related to requirements
 - Plan-driven methods hope to manage risk related to design

Definition of Process

- A process meets the goals of a project by:
 - Orchestrating activities to achieve the goal
 - Defining artifacts to be produced by the activities
 - Determining the roles and skills needed
 - Setting criteria for progressing from an activity
 - Specifying events for planning and tracking

Common types of activities

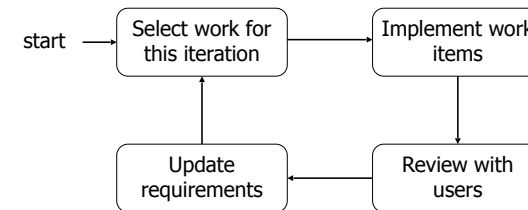
- Analisi dei requisiti (specifiche)
- Progettazione (design)
- Implementazione (codifica)
- Convalida (testing e approvazione)
- Rilascio e manutenzione operativa (Deployment & Maintenance)

Terminology

- Process Model:
 - A class of processes; e.g., Iterative Model, Waterfall Model
- Framework:
 - Partial specification of a process class; e.g., Scrum, Spiral
- Practice:
 - Specific actionable descriptions of an activity

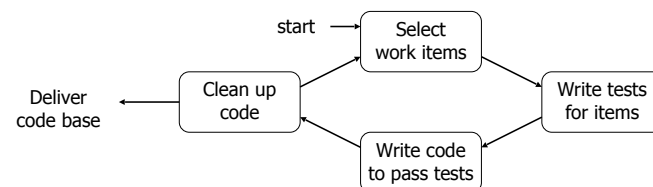
An Iterative Process

- Typically, diagrams show only the activities (as boxes)
 - This particular process shows what is to be done, not how to do it

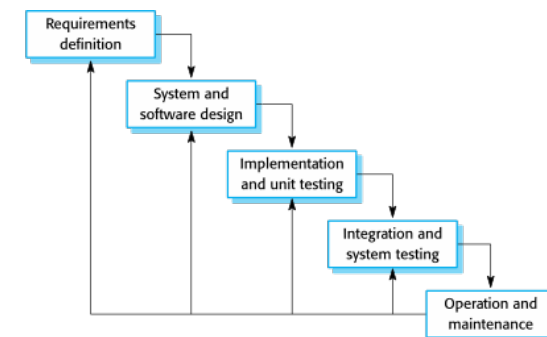


What Does this process do?

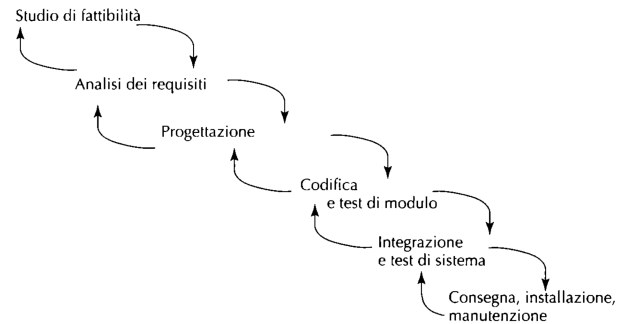
- The diagram has 4 activities and 1 event. Identify the event.



Cascata (Waterfall) [Royce 1970]



Waterfall con feedback



Waterfall

Processo statico con tanta documentazione

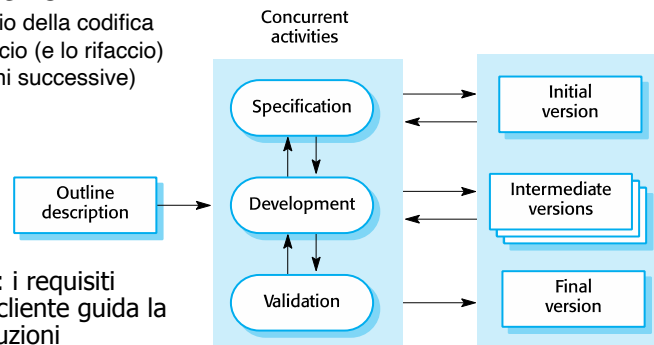
- Lungo tempo per ottenere il prodotto
- Poche interazioni con i clienti (solo nella fase iniziale)
- Difficoltà ad introdurre i cambiamenti richiesti dal cliente

- + Consistenza tra artefatti
- + Ampia documentazione
- + Utile se i requisiti sono stabili e chiaramente definiti
- + Adatto a sistemi grandi, complessi, critici, per gestire team numerosi
- + Alta qualità del codice prodotto

Processo Evolutivo

Non voglio ritardare l'inizio della codifica dell'intero sistema: lo faccio (e lo rifaccio) **per intero** (trasformazioni successive)

- Variante esplorativa: i requisiti iniziali ben chiari, il cliente guida la sequenza delle evoluzioni
- Build & Fix: documentazione quasi inesistente, bassa qualità



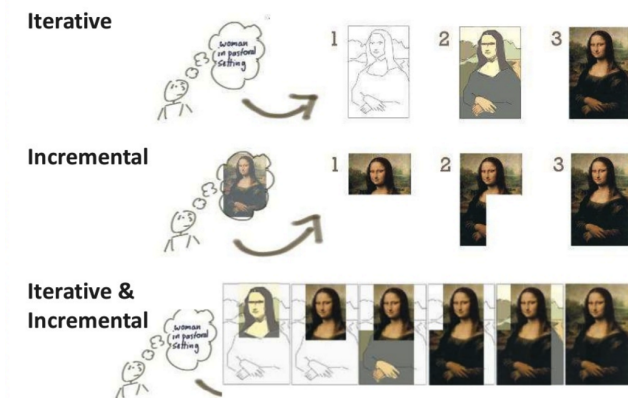
Evolutivo

- Problemi
 - Tempi lunghi
 - Sistemi difficilmente comprensibili e modificabili, probabilmente non corretti
 - Mancanza di visione d'insieme del progetto
- Applicabilità
 - Sistemi di piccole dimensioni
 - Singole parti di sistemi grandi (es. interfaccia utente)
 - Sistemi con vita breve (es. prototipi)

Altri Processi

- Processo di Sviluppo Incrementale
 - Sono implementate prima le funzionalità di base (o prioritarie)
 - Al codice sviluppato in precedenza è aggiunto altro codice per un altro gruppo di funzionalità
 - Si ripete il passo precedente, fino a completamento
- Processo CBSE o basato su COTS
 - COTS = componenti esistenti (Components Off The Shelf)
 - Analisi dei componenti esistenti
 - Modifica dei requisiti (?)
 - Progettazione tramite riuso
 - Sviluppo (di adattatori) ed integrazione

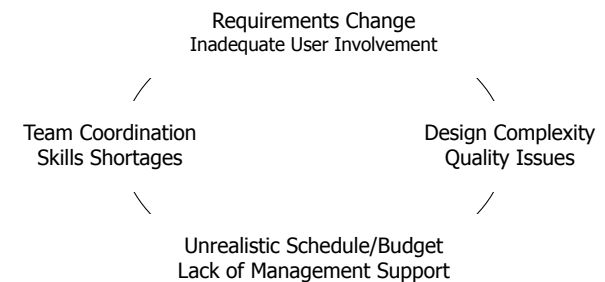
Iterativo vs Incrementale



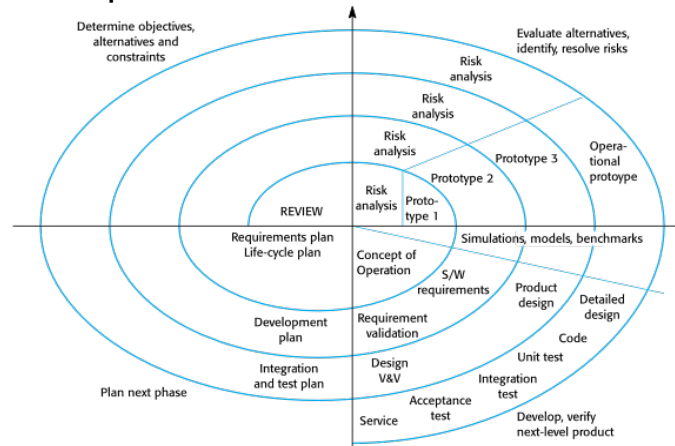
Project Risks

- What are the risks, if any, for your project, related to:
 - Users and their needs
 - Team skills and coordination
 - Technology
 - Constraints; e.g., schedule, resources

Sample Project Risks



Modello a Spirale



Settori del processo a Spirale

- Stabilire obiettivi
 - Gli obiettivi per la fase corrente sono identificati
- Valutare il rischio e ridurlo
 - I rischi sono valutati ed attività sono intraprese per ridurre quelli più importanti
- Sviluppo e convalida
 - Secondo uno dei modelli precedenti
- Pianificazione
 - Il progetto è revisionato e la prossima fase della spirale è pianificata

Agile (Software Development)

- The term was coined at a February 2001 meeting
 - By self-described "independent thinkers about software development"
- The same meeting resulted in the Agile Manifesto
 - Attendees included Ken Schwaber and Jeff Sutherland who created Scrum
 - And Kent Beck, responsible for Extreme Programming (XP)
 - Plus proponents of several other agile methods
- Developers have the flexibility and accountability for doing the work

Contrasting Agile and Plan-Driven Development

- | | |
|---|--|
| <ul style="list-style-type: none"> • Agile Manifesto <ul style="list-style-type: none"> – Individuals and interactions – Working Software – Customer Collaboration – Responding to Change | <ul style="list-style-type: none"> • Plan Driven (Plan then Build) <ul style="list-style-type: none"> – Processes and tools – Comprehensive documentation – Contract negotiations – Following a plan |
|---|--|

What is an Agile Method?

- The Agile Alliance's definition of an agile method:
 - It's agile if it conforms to the Agile Manifesto and the principles behind it
- Scrum and XP are pre-Manifesto agile methods
 - Both date back to the mid 1990s

Agile Development with XP



Extreme Programming (XP)

- Kent Beck defined XP in the mid-1990s, while on a project for Chrysler
- The name comes from taking common-sense practices to the extreme:
 - Example: "If design is good, we'll make it part of everybody's daily business (refactoring)."
- XP was refined in the years following the Chrysler project
 - There are now XP values, principles, activities, and practices

Overview of XP

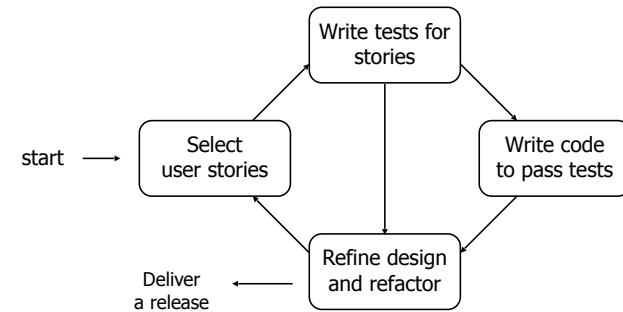
- Beck describes XP as follows:
 - "an always deployable system to which features, chosen by the customer, are added and automatically tested on a fixed heartbeat."
- The "heartbeat" or iteration length is often a week or two (mai oltre 4)

XP key points

- XP focalizza sul codice
 - Fare solo le cose che sveltiscono la produzione del codice
 - Codifica e test
- XP si orienta sulla gente
 - La conoscenza del sistema è trasferita attraverso la comunicazione tra la gente
- XP è leggero
 - Rimuovere i costi aggiuntivi
 - Creare prodotti di qualità tramite test rigorosi

The Test-Driven XP Development Cycle

User stories express requirements, as we'll see



Note that the XP cycle allows an iteration to bypass coding

The Four Main XP Activities

- Listening
 - Testing
 - Coding
 - Design
-
- XP teams can face design risk if they defer design until the last minute
 - When to Design remains an issue

Four Main Activities

- Listening: to what users want from a system
 - Testing: Make it central to development
 - Coding: Write simple readable code
 - Design: Refactor to clean up the design and implementation
-
- These activities come together in the XP development cycle (next slide)

Listening: Writing User Stories (Requirements)



User Stories are for Snippets of Functionality

Write from the user's perspective

- Connextra User Story Template
- As a <role> I want to <action> so that <benefit>
- Example:
 - As a customer,
 - I want to withdraw cash from an ATM
 - so that I can get cash when the bank is closed

Examples: Stories for a Ride-Sharing Service

- Example 1:
 - As a rider
 - I want flexible on-demand transportation
 - so that I can get to my destination whenever I want
- Example 2:
 - As a rider
 - I want a trusted driver
 - so that I will feel safe getting into the driver's vehicle

General Guidelines for User Stories

Purpose is identifying a user requirement

- Keep it small enough to fit on a 3x5 inch index card
- Use simple plain language, from the user's point of view
- Write about a user need, not about technology to meet the need
- Use the template: As a <role> I want to <action> so that <benefit>
 - Confirm the story by writing one or more acceptance tests

Story Card

Customer Story and Task Card Blw Development / COLA

DATE: 3/19/91 TYPE OF ACTIVITY: NEW ☒ FIX: ☐ ENHANCE: ☐ FUNC. TEST: ☐

STORY NUMBER: 1275 PRIORITY: USER: TECH: ☐

PRIOR REFERENCE: RISK: TECH ESTIMATE:

TASK DESCRIPTION:
 SPLIT COLA: When the COLA rate chgs. in the middle of the Blw Pay Period, we will want to pay the 1st week of the pay period at the OLD COLA rate and the 2nd week of the pay period at the NEW COLA rate. Should occur automatically based on system design.

NOTES:
 For the OT, we will run a m/frame program that will pay or calc the COLA on the 2nd week of OT. The plant currently retransmits the hours data for the 2nd week exclusively so that we can calc COLA. This will come into the Model as a "2144" COLA.

TASK TRACKING: Gross Pay Adjustment, Create RM Boundary and Place in DEE+Exp+COLA

Date	Status	To Do	Comments

Rather than a lengthy requirements document, the customer writes user stories, which define the functionality the customer would like to see, along with the business value and **priority of each of those features.**

With XP , Testing Plays a Central Role

Test driven development (TDD)



Test-Driven Development

Dates back to NASA's Project Mercury in the 1960s



- Write acceptance tests before coding
 - The tests should fail for the right reasons
- Then, write just enough code to pass the tests
 - Add unit tests to verify the correctness of the new code
- Refactor to clean up the design and the code

Testing Supports All Activities

- Three forms of testing
 - Acceptance tests for user stories
 - Unit tests for the code
 - Regression tests for guarding against breaking existing functionality
- Testing after every change increases confidence in the code
 - Aids debugging: either the change was faulty or it broke something
 - Such Testing can actually shorten overall development time

Validation and Verification

- Validation is for functionality and requirements
 - The focus is on “Am I building the right product?”
- Verification is for implementation correctness
 - The focus is on “Am I building the product right?”
- During test-driven development
 - Before coding, write acceptance tests for validation
 - After coding, write additional unit tests for verification

Acceptance-Test Template for User Stories

- In effect, acceptance tests define what a product does
- Simple form of the template:
 - Given <some context> when <triggering event> then <expect outcome>
- There may be multiple acceptance tests for a user story
- Write acceptance tests so they are executable

General Form of the Acceptance Template

- Given <some initial context>
 - and <some more context>
- when <triggering event>
- then <expect outcome>
- and <another outcome>
- There may be multiple and-clauses for context and outcomes

Example: Case 1 for Cash Withdrawal

Account is in Credit

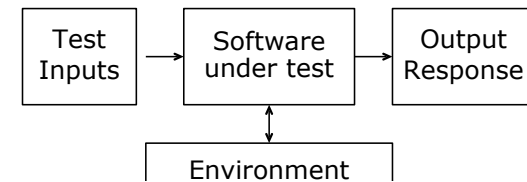
- Given the account is in credit
 - and the card is valid
 - and the dispenser has cash
- when the customer requests cash
- then update account balance
 - and dispense cash
 - and return card

Example: Case 2

Account is Overdrawn Past the Credit Limit

- Given the account is overdrawn
- and the card is valid
- when the customer requests cash
- then display "sorry" message
- and return card

Relating the Acceptance Template to Testing



- The "Given" context sets up the environment
- The "when" event corresponds to an input
- The "then" response corresponds to the expected output

Coding: Write Simple Readable Code



Refactoring: Code Clean UP and Restructuring

Refactoring consists of a sequence of correctness-preserving changes

- Restore modularity, so each module has a specific responsibility
- Put common (duplicated) functionality into a single method or module
- Improve control flow for readability and predictability of behavior
- Ensure that variable and method names reflect their purpose
- Add comments for rationale and what not to change

Design: When and How Much?



When to Design?

The answer varies from project to project

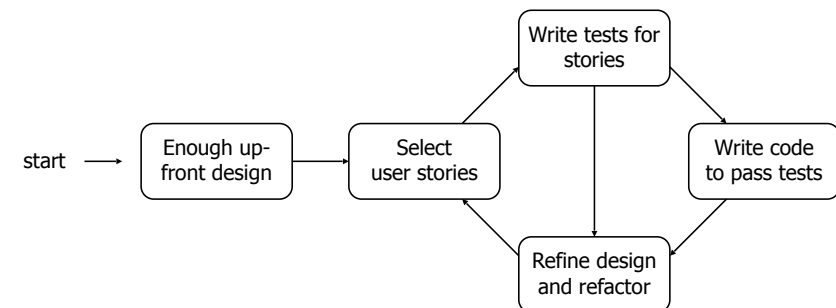
- Too much design too soon
 - Can lead to wasted effort on something that will not be needed
- Too little design too late
 - Can result in an unwieldy design that needs rework
- How do we avoid either wasted effort or poor design?

Early Failures Due to Last Minute Design

From the 2nd edition of Extreme Programming Explained by Beck and Andres

- Kent Beck writes that early XP teams misinterpreted the 1st edition
 - They deferred design until the last possible moment
 - And ended up with brittle poorly designed systems
- He recommends deferring design until the “last responsible moment”
 - “Keep design investment in proportion to the needs of the system”

Modified XP Cycle with Some Up-Front Design



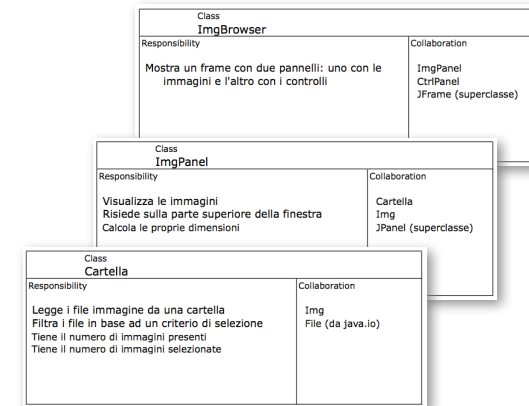
When to Design Depends on Product and Requirements

Specifically, on product complexity and requirements stability

PRODUCT	Complex	Design properties into the system up front	Start with a modifiable modular design
	Simple	Minimal initial design to set direction	Defer major decisions until needed
		Stable	Changing
		REQUIREMENTS	

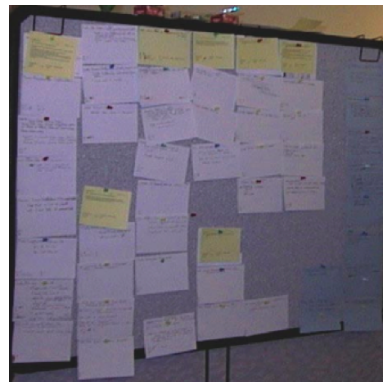
CRC cards

- Class Responsibility Collaboration
- Documentano l'architettura in XP
- Aiutano a discuterne e ragionarci
- Molto più informale e sintetico di un class diagram



Planning Game

- Il cliente scrivono le **storie** (i requisiti) e gli assegnano **priorità in termini di valore**, gli sviluppatori stimano il **tempo** di sviluppo
- Il **cliente** riempie da due **fino a 3-4 settimane** di lavoro scegliendo le storie (aggiornabili): per coprire almeno la prossima iterazione
- Ad ogni iterazione, **gli sviluppatori**: dividono ciascuna storia in **task**, ne valutano difficoltà (rischio) e tempo di sviluppo
- Dando **priorità ai task più rischiosi**, ciascuno si impegna per **realizzare** un task
- Il cliente decide le date per le varie main release (1-3 mesi)



Story Board

12 Pratiche di XP

- Gioco di pianificazione: users vs dev.s
- Piccole release: tipicamente una ad iterazione
- Usare Metafore: nomi significativi aiutano a capire e comunicare in modo immediato
- Testing continuo: anche più volte al giorno
- Refactoring: per design e debito tecnico
- Pair Programming: static review, test
- Cliente in sede: comunicazione continua in tempo reale, collabora e vede i progressi
- Design semplice: cambiarlo deve essere sempre possibile
- Possesso del codice collettivo: tutti sono responsabili di tutto
- Integrazione continua: non meno di una volta al giorno
- Settimana di 40 ore: evitare rischio stress
- Usare gli standard per il codice: migliora la leggibilità

<http://java.sun.com/docs/codeconv/html/CodeConvTOC.doc.html>

Riepilogo processi/metodi

- Waterfall (processo di sviluppo rigidamente sequenziale, one-pass)
- Evolutionary (una versione iniziale si trasforma, via via, in quella finale)
- Incremental (partiziono lo sviluppo) vs Iterativo puro (approfondisco lo svil.)
- Component Based (massimizzo il riuso, minimizzo il coding)
- Spiral (è un framework, iterativo, pianifico per ridurre il rischio di cattivo design)
- Agile (è una metodologia, suggerisce processi "leggeri" per agevolare i cambiamenti)
- XP, Scrum, sono metodi agili (e incrementali)

Pair Programming

- Programmatori esperti e motivati
- Ruolo di uno dei partner, driver
 - Usa il mouse e la tastiera
 - Pensa al miglior modo di implementare il metodo
- Ruolo dell'altro, navigator
 - L'approccio funzionerà? Revisione
 - Pensa ai test
 - Potrebbe essere fatto più semplicemente?



Pair Programming Remains Controversial

- Pair programming is two people on the same task at the same screen
 - The proposed benefit is that code is reviewed all the time
 - Also, if someone leaves, there is a person who knows the code
- The practice has not caught on
 - Incompatibilità caratteriali, antipatie

12 Pratiche di XP

- Gioco di pianificazione
- **Piccole release**
- Metafora
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- **Metafora**
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- **Testing**
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- **Refactoring**
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- **Pair Programming**
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- Pair Programming
- **Cliente in sede**
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- **Design semplice**
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- **Possesso del codice collettivo**
- Integrazione continua
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- **Integrazione continua**
- Settimana di 40 ore
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- **Settimana di 40 ore**
- Usare gli standard per il codice

12 Pratiche di XP [Beck]

- Gioco di pianificazione
- Piccole release
- Metafora
- Testing
- Refactoring
- Pair Programming
- Cliente in sede
- Design semplice
- Possesso del codice collettivo
- Integrazione continua
- Settimana di 40 ore
- **Usare gli standard per il codice**

Refactoring



Refactoring

- Refactoring è il processo che cambia un sistema software in modo che il comportamento esterno del sistema non cambi, ovvero i requisiti funzionali soddisfatti sono gli stessi, per far sì che la **struttura interna sia migliorata**
- Si migliora la progettazione a poco a poco, durante e dopo l'implementazione del codice
- Esempio semplice: consolidare (ovvero eliminare) frammenti di codice duplicati all'interno di rami condizionali
- Per passare dalla versione 0.2 alla versione 0.9 del programma presentato nelle prime lezioni (calcola Importi) è stata usata la tecnica di Refactoring Estrai Metodo

Refactoring

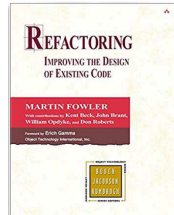
- L'idea del refactoring è di riconoscere che è difficile ottenere fin dall'inizio una progettazione adeguata (e un codice adeguato) e quindi si deve cambiare
- Questi cambiamenti permettono quindi di incorporare più facilmente nuovi requisiti, o requisiti che cambiano
- Il refactoring fornisce tecniche per evolvere la progettazione in piccoli passi
- Vantaggi
 - Spesso la dimensione del codice si riduce dopo il refactoring
 - Le strutture complicate si trasformano in strutture più semplici più facili da capire e mantenere
 - Si evita di introdurre un debito tecnico all'interno della progettazione

Perché fare refactoring

- Migliora la progettazione del sistema software
Senza refactoring la progettazione si deteriora mano a mano che si apportano modifiche al codice
- Rende il codice più facile da capire, perché la struttura del codice viene migliorata, il codice duplicato rimosso, etc.
- Aiuta a scoprire bug, fare refactoring promuove la comprensione profonda del codice e questo...
- Permette di programmare più velocemente, perché una buona progettazione è più facile da cambiare

Come si fa refactoring

- Usare tecniche di refactoring note [Fowler]
- Fare test costantemente: prima scrivere i test, dopo fare refactoring e verificare che tutti i test siano ancora superati
- Se un test non è superato, il refactoring ha compromesso qualcosa che funzionava, si è subito avvertiti e bisogna subito intervenire
- [Fowler] M.Fowler. Refactoring: Improving the Design of Existing Code. Addison-Wesley



Estrai metodo

- Un frammento di codice può essere estratto.
- Far diventare quel frammento un metodo il cui nome spiega lo scopo del frammento

```
public void printDovuto(double amount) {  
    printBanner();  
    System.out.println("nome: " + nome);  
    System.out.println("tot: " + somma);  
}
```

Diventa

```
public void printConto(double somma) {  
    printBanner();  
    printDettagli(somma);  
}  
  
public void printDettagli(double amount) {  
    System.out.println("nome: " + nome);  
    System.out.println("tot: " + amount);  
}
```

Estrai Metodo: motivazioni

- Cercare metodi lunghi o codice che necessita di commenti per essere comprensibile
- Ridurre la lunghezza dei metodi, poiché metodi piccoli sono con più probabilità utilizzabili richiamandoli da altri metodi
- Scegliere un buon nome per i metodi piccoli, che esprime ciò che il metodo fa
- I metodi di alto livello, quelli che invocano i metodi piccoli, sono più facili da comprendere, sembrano essere costituiti da una sequenza di commenti
- Fare overriding (ridefinire nella sottoclasse) è più semplice se si hanno metodi piccoli

Estrai Metodo: meccanismi

- Creare un nuovo metodo il cui nome comunica l'intenzione del metodo
- Copiare il codice estratto dal metodo sorgente al nuovo metodo
- Guardare se il codice estratto ha variabili locali al metodo sorgente e far diventare tali variabili parametri del metodo nuovo
- Se alcune variabili sono usate solo all'interno del codice estratto farle diventare variabili temporanee del nuovo metodo
- Se una variabile locale al metodo sorgente è modificata dal codice estratto, vedere se è possibile far sì che il metodo estratto sia una query e assegnare il risultato alla variabile locale del metodo sorgente
- Sostituire nel metodo sorgente il codice estratto con una chiamata al metodo nuovo

Esempio Estrai Metodo

```
public class Ordine {
    private double importo;
    public double getImporto() {
        return importo;
    }
}

public class Cliente {
    private ArrayList<Ordine> ordini;
    private String nome = "Mike";

    public void printDovuto() {
        Iterator<Ordine> i = ordini.iterator();
        double tot = 0;
        // scrivi banner
        System.out.println("-----");
        System.out.println("- Cliente Dare -");
        System.out.println("-----");
        // calcola totale
        while (i.hasNext()) {
            Ordine o = i.next();
            tot += o.getImporto();
        }
        // scrivi dettagli
        System.out.println("nome: " + nome);
        System.out.println("tot: " + tot);
    }
}

public class Cliente {
    private ArrayList<Ordine> ordini;
    private String nome = "Mike";

    public void printDovuto() {
        printBanner();
        double tot = getTotale();
        printDettagli(tot);
    }

    public double getTotale() {
        Iterator<Ordine> i = ordini.iterator();
        double tot = 0;
        while (i.hasNext()) {
            Ordine o = i.next();
            tot += o.getImporto();
        }
        return tot;
    }

    public void printBanner() {
        System.out.println("-----");
        System.out.println("- Cliente Dare -");
        System.out.println("-----");
    }

    public void printDettagli(double somma) {
        System.out.println("nome: " + nome);
        System.out.println("tot: " + somma);
    }
}
```

Sostituisci temp con query

- Una variabile temporanea (temp) è usata per tenere il risultato di un'espressione
- Estrarre l'espressione ed inserirla in un metodo (query).
- Sostituire tutti i riferimenti a temp con la chiamata al metodo che incapsula l'espressione.
- Il nuovo metodo può essere richiamato anche altrove

```
double prezzoBase = quantita * prezzo;
```

Diventa

```
private double prezzoBase() {
    return quantita * prezzo;
}
```

Sostituisci T con Q: motivazioni

- Le variabili temp sono temporanee e locali. Sono visibili solo all'interno del contesto locale e per questo inducono ad avere metodi lunghi, perché è il solo modo per tenere in vita e continuare a usare quel dato conservato.
- Con la sostituzione effettuata tramite il refactoring, qualsiasi metodo può avere il dato
- Spesso si effettua il refactoring Sostituisci Temp con Query prima di Estrai Metodo
- Questo refactoring è facile se temp è assegnata solo una volta
 - Altrimenti applica prima Split temp var

Sostituisci T con Q: meccanismi

- Cc Cercare una variabile temporanea assegnata solo una volta
- Dichiarare temp final
- Compilare (per verificare che è assegnata una volta sola)
- Estrarre la parte destra dell'assegnazione e creare un metodo

Esempio Sostituisci T con Q

```
private double quantita, prezzo;

public double getPrezzo1() {
    double prezzoBase = quantita * prezzo;
    double sconto;
    if (prezzoBase > 1000) sconto = 0.95; else sconto = 0.98;
    return prezzoBase * sconto;
}
```

Diventa, sostituendo entrambe le variabili prezzoBase e sconto

```
private double quantita, prezzo;

public double getPrezzo2() { return prezzoBase() * sconto(); }

private double prezzoBase() { return quantita * prezzo; }

private double sconto() {
    if (prezzoBase() > 1000) return 0.95;
    return 0.98;
}
```

Dividi Variabile temp

- Una variabile temporanea è assegnata più di una volta, ma non è una variabile assegnata in un loop o usata per accumulare valori
- Usa una variabile separata per ciascun assegnamento

```
double temp = 2 * (height + width);
System.out.println(temp);
temp = height * width;
System.out.println(temp);
```

• Diventa

```
final double perim = 2 * (height + width);
System.out.println(perim);
final double area = height * width;
System.out.println(area);
```

Dividi Var Temp: motivazioni

- Le variabili temporanee (temp) hanno vari usi. Alcuni usi portano ad assegnare temp più volte, es. variabili che cambiano ad ogni passata di un ciclo (queste assegnazioni multiple sono ok)
- Variabili temp che tengono il risultato di un metodo lungo, per essere usate dopo, dovrebbero essere assegnate una volta soltanto
- Se sono assegnate più di una volta allora hanno più di una responsabilità all'interno del metodo.
- Ogni variabile dovrebbe avere una sola responsabilità e dovrebbe essere sostituita con una temp per ciascuna responsabilità
- Usare una stessa temp per due cose diverse confonde il lettore

Dividi Var Temp: meccanismi

- Cambiare il nome della variabile temp al momento della dichiarazione e alla sua prima assegnazione
- Dichiarare la nuova temp come final
- Cambiare tutti i riferimenti a temp fino alla sua seconda assegnazione
- Dichiarare una nuova temp per la seconda assegnazione
- Compilare e testare
- Ripetere per singoli passi, ogni passo rinomina una dichiarazione e cambia i riferimenti fino alla prossima assegnazione

Esempio Dividi Var temp

```
private double primaryForce, secondaryForce, mass, delay;
public double getDistanceTravelled1(int time) {
    double result;
    double acc = primaryForce / mass; // prima assegnazione
    int primaryTime = (int) Math.min(time, delay);
    result = 0.5 * acc * primaryTime * primaryTime;
    int secondT = (int) (time - delay);
    if (secondT > 0) {
        double primaryVel = acc * delay;
        acc = (primaryForce + secondaryForce) / mass; // seconda assegnazione
        result += primaryVel * secondT + 0.5 * acc * secondT * secondT;
    }
    return result;
}

DIVENTA:
public double getDistanceTravelled2(int time) {
    double result;
    final double primAcc = primaryForce / mass;
    int primaryTime = (int) Math.min(time, delay);
    result = 0.5 * primAcc * primaryTime * primaryTime;
    int secondT = (int) (time - delay);
    if (secondT > 0) {
        double primaryVel = primAcc * delay;
        final double secondAcc = (primaryForce + secondaryForce) / mass;
        result += primaryVel * secondT + 0.5 * secondAcc * secondT * secondT;
    }
    return result;
}
```

Esercitazione (a casa)

- Ristrutturare il codice del Binary Search Tree per introdurre al suo interno i pattern di refactoring presentati
- Scrivere alcuni programmi che testano le funzionalità del modulo per verificarne la correttezza prima e dopo il refactoring

The Scrum Framework



Scrum Principles

- Scrum focuses on **teamwork**
 - It has rules for iteration planning and progress reviews
- The Framework does not tell developers how to do their work
 - XP focuses on how to do the work
- Scrum è basato **sull'empirismo** e sul concetto di **lean (snello)**
- Recommendation: Use Scrum together with XP
 - Within the planning and review structure of Scrum, use XP for the work

81% of Agile Projects Use Scrum or a Scrum Hybrid

- 66% use Scrum by itself
 - How developers do their work is not specified
- 15% use a Scrum hybrid with another method
 - 9% ScrumBan (Scrum+Kanban)
 - 6% Scrum+XP

Structure Provided by Scrum

- Development is keyed to iterations, called **sprints**
- planning and review **events** during a sprint
 - **Daily Scrum** and **Sprint Review**
 - Sprints and events are **"time boxed"**; that is, they are time limited
- Scrum also specifies **roles** and **artifacts**
- Scrum pillars: **Transparency -> Inspection -> Adaptation**

The Scrum Team: Roles

The **product owner** represents the voice of the customer

- Manages the **product backlog** of work items (requirements)
- Sets the **goal** for what to accomplish during a **sprint**

Developers choose how they implement work items

- Review events hold them accountable
- deliver** potentially shippable product "**increments**" per sprint

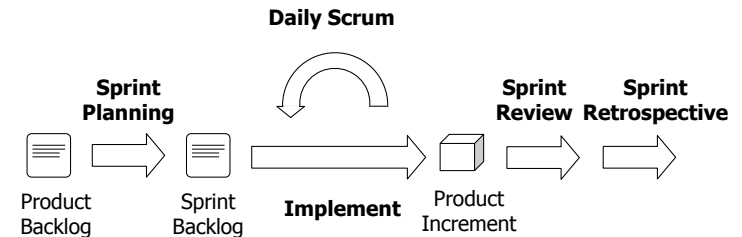
Scrum Master is a coach (servant-leader): un moderatore e aiuto

- Organizes** and moderates all **events**, respectfully keeping events on track
- Additional responsibility during Daily Scrums: **removing external impediments**

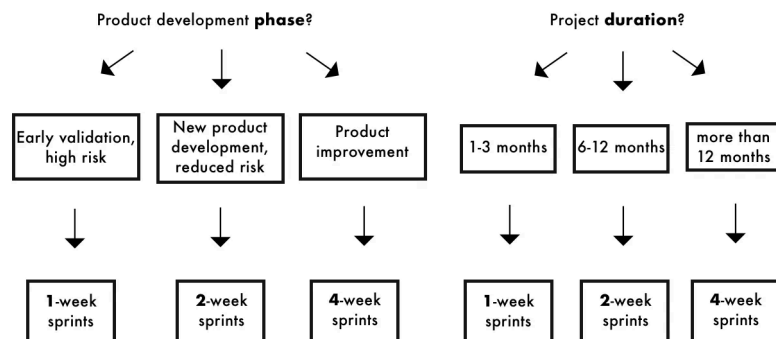
•

Overview of a Sprint: Events and Artifacts

Arrows show events, with one exception: Implement is not a Scrum event

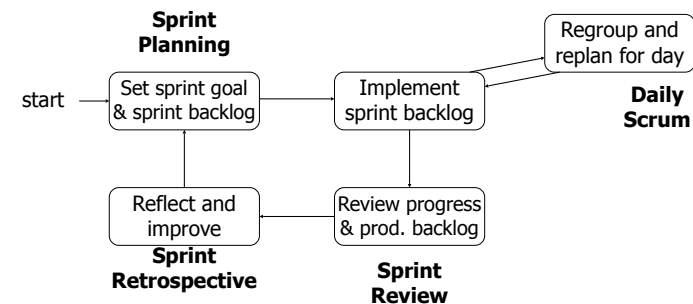


Optimal Sprint Duration



Activities Associated with Scrum Events

Activities are represented by boxes; events appear as labels next to activities



Events: Sprint Planning

- Attendees: The Scrum Team and users
- Max two hours/week * sprint weeks
- Led by product owner, **set a Sprint Goal**, which then cannot be changed
- Selects a **Sprint Backlog** of work items to meet the goal
 - Users define the value/**priority** of backlog items
 - Developers estimate **effort** for work items and **what** will fit in a sprint

Events: Daily Scrum

- Attendees: Developers, with Scrum Master as moderator
- Max 15 min.
- Each developer answers 3 questions:
 - What did I do yesterday?
 - What will I do today?
 - Any obstacles?
- The Scrum Master takes responsibility for removing any external obstacles

Example: Removing External Obstacle

- Obstacle: Developers find a bug in a video subsystem from a supplier
 - The bug could delay the project
- The Scrum Master takes responsibility for removing the obstacle
 - The Scrum master works with the purchasing manager
 - Together, they contact the supplier
 - The contract is large, so the supplier expedites a fix

Events: Sprint Review

- Attendees: Scrum Team and users
- Review product increment for this sprint
- Update Product Backlog based on user feedback
- Max 4 hours for a four months project

Events: Sprint Retrospective

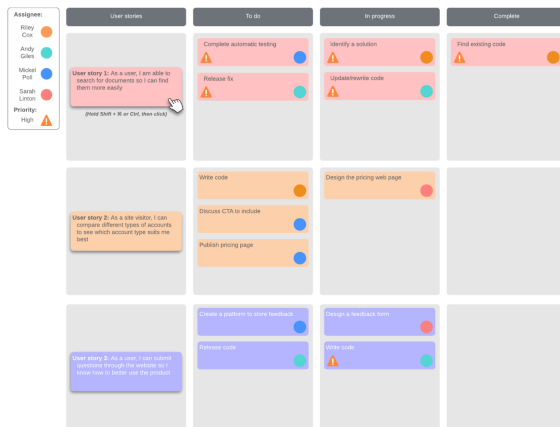
- Attendees: Scrum Team
 - Plan **process** improvements for the next Sprint
 - This event is not about the product
- Max 3 hours for 4 months project

Scrum Artifacts

- Product Backlog:** Ordered list of **all work items** that might be needed for a product
- Sprint Backlog:** Items **selected** for meeting the Sprint Goal that is set during planning
- Product Increment:** Sum of all Product Backlog items that are **completed** during a Sprint

For transparency of information, artifacts are open to all team members

Example Scrum Board



Shows current Sprint goals (user stories) and respective sprint backlog

Task move from left to right

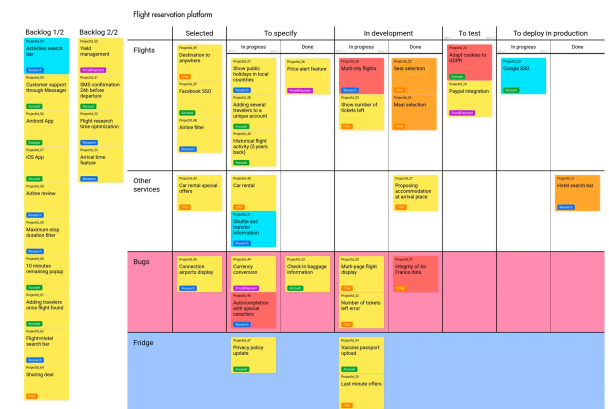
Example Kanban Board

visualize and track all work **in bird view**

rappresenta lo stato di avanzamento complessivo

Kanban boards are much more fluid and can be more easily adapted.

Kanban has no practice "rules"



What if the Sprint Backlog is Too Optimistic?

- Situation:
 - Developers overestimate what they can accomplish during a sprint
 - That is, they realize that they cannot implement the full Sprint Backlog
- Response:
 - Developers can **renegotiate** the Sprint Backlog with the Product Owner
 - The Sprint Goal cannot be changed

How Does Scrum Scale?

- Situation:
 - The development team is too large for a 15-minute Daily Scrum
 - With more than 7 developers, 15 minutes means less than 2 per person
- Response:
 - On approach is to **split** the team into subteams
 - Each subteam has its own Scrum Masters
 - The Scrum Masters from the subteams then have a Scrum of Scrums

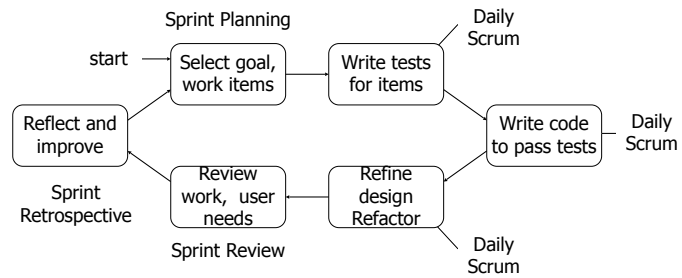
What if Some Features Don't Fit?

- Situation:
 - The entire Product Backlog cannot be done during the project schedule
- Response:
 - With any iterative project, do **highest priority** features during a sprint
 - **Inessential features** that don't fit within the schedule **can be dropped**
 - If essential features don't fit, **renegotiate** with users as early as possible
 - Avoid surprises by involving users in planning, from the start

Recommendations

- Use Scrum
- Adopt practices from XP – they fit seamlessly with Scrum
- Values drive XP's development practices
 - Kent Beck also created xUnit so there would be no excuse for not testing
- Early XP teams faced design risk
 - They did too little design too late

A Scrum/XP Hybrid Process



Recap

XP vs Scrum

- Shorter iterations
- Flexible with the changes
- Focus on technical practices
- Customer determines the order of feature development
- Longer sprints
- No changes within sprints
- Focus on managerial aspects
- Self-organized teams that decide what features to work on first

Software Quality Metrics



Software change

- Software change is inevitable
 - New requirements emerge when the software is used;
 - The business environment changes;
 - Errors must be repaired;
 - New computers and equipment is added to the system;
 - The performance or reliability of the system may have to be improved.
- A key problem for organisations is implementing and managing change to their existing software systems.

Software maintenance

- Modifying a program **after** it has been put into use.
- Maintenance does not normally involve major changes to the system's architecture.
- Changes are implemented by **modifying** existing components and **adding** new **components** to the system.
- Studies have shown that most maintenance effort is spent on a relatively small number of system components.

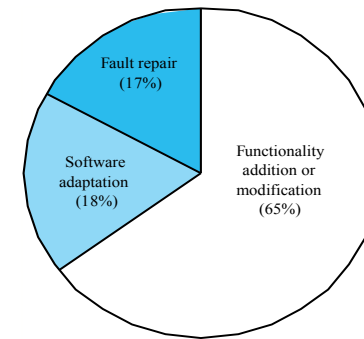
Maintenance is inevitable

- The system requirements are likely to **change while** the system is being **developed** because the environment is changing. Therefore a delivered system won't meet its requirements!
- Systems are tightly coupled with their environment. When a system is installed in an environment **it changes that environment** and therefore changes the system requirements.
- Systems **MUST** be maintained therefore if they are **to remain useful** in an environment.

Types of maintenance

- Maintenance to **repair** software faults
 - Changing a system to correct deficiencies in the way meets its requirements.
- Maintenance to **adapt** software to a different operating environment
 - Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.
- Maintenance to **add** to or **modify** the system's functionality
 - Modifying the system to satisfy new requirements.

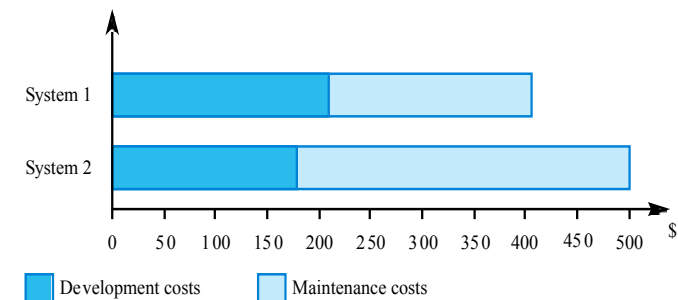
Distribution of maintenance effort



Maintenance costs

- Usually greater than development costs (2* to 100* depending on the application).
- Affected by both technical and non-technical factors.
- Increases as software is maintained. **Maintenance corrupts** the software structure so makes further maintenance more difficult.
- Ageing software can have high support costs (e.g. old languages, compilers etc.).

Development/maintenance costs



Program evolution dynamics

- Program evolution dynamics is the study of the processes of system change.
- After major empirical studies, Lehman and Belady proposed that there were a number of 'laws' which applied to all systems as they evolved.
- There are sensible observations rather than laws. They are applicable to large systems developed by large organisations. Perhaps less applicable in other cases.

Lehman's laws

Law	Description
Continuing change	A program that is used in a real-world environment necessarily must change or become progressively less useful in that environment.
Increasing complexity	As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.
Large program evolution	Program evolution is a self-regulating process. System attributes such as size, time between releases and the number of reported errors is approximately invariant for each system release.
Organisational stability	Over a program's lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.
Conservation of familiarity	Over the lifetime of a system, the incremental change in each release is approximately constant.
Continuing growth	The functionality offered by systems has to continually increase to maintain user satisfaction.
Declining quality	The quality of systems will appear to be declining unless they are adapted to changes in their operational environment.

Applicability of Lehman's laws

- Lehman's laws seem to be generally applicable to large, tailored systems developed by large organisations.
- It is not clear how they should be modified for
 - Shrink-wrapped software products;
 - Systems that incorporate a significant number of COTS components;
 - Small organisations;
 - Medium sized systems.

Maintenance cost factors

- Team stability
 - Maintenance costs are reduced if the same staff are involved with them for some time.
- Contractual responsibility
 - The developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change.
- Staff skills
 - Maintenance staff are often inexperienced and have limited domain knowledge.
- Program age and structure
 - As programs age, their structure is degraded and they become harder to understand and change.

Motivation for metrics

- Estimate the cost & schedule of future projects
- Evaluate the productivity impacts of new tools and techniques
- Improve software quality
- Forecast future staffing needs
- Anticipate and reduce future maintenance needs

Complexity metrics

- Predictions of maintainability can be made by assessing the complexity of system components.
- Studies have shown that most maintenance effort is spent on a relatively small number of system components.
- Complexity depends on
 - Complexity of control structures;
 - Complexity of data structures;
 - Object, method (procedure) and module size.

How do you measure data complexity?

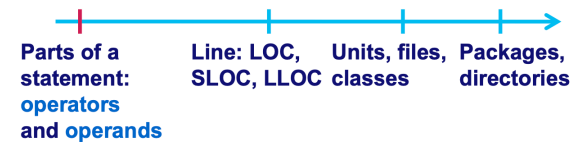
Data Complexity Measures

1. Measures of overlaps in feature values from different data classes.
2. Measures of separability of classes.
3. Measures of geometry, topology, and density of manifolds.

Complexity analysis is a technique to characterise the time taken by an algorithm with respect to input size (independent from the machine, language and compiler).

Complexity Metrics: measuring size

- **Sometimes is classified as size rather than complexity**
- **Unit of measurement**



- **Operators:**
 - traditional (+, ++, >), keywords (return, if, continue)
- **Operands**
 - identifiers, constants

LOC/SLOC

Dimensione

LOC o SLOC (lines of source code), oppure NCNB (non comment non blank)

"A line of code is **any line** of program text that is **not a comment or blank** line, regardless of the number of statements or fragments of statements on the line. This specifically includes all lines containing program headers, declarations, and executable and non-executable statements."

Comment Percentage

Percentuale di commenti rispetto a LOC (consigliato 30%)

Halstead metrics (1977)

• Four basic metrics of Halstead

	Total	Unique
Operators	N1	n1
Operands	N2	n2

- Length: $N = N1 + N2$
- Vocabulary: $n = n1 + n2$
- Volume: $V = N \log_2 n$
 - Insensitive to lay-out
- VerifySoft:
 - $20 \leq \text{Volume}(\text{function}) \leq 1000$
 - $100 \leq \text{Volume}(\text{file}) \leq 8000$

Halstead metrics: Example

```
void sort ( int *a, int n ) {  
  int i, j, t;  
  
  if ( n < 2 ) return;  
  for ( i=0 ; i < n-1; i++ ) {  
    for ( j=i+1 ; j < n ; j++ ) {  
      if ( a[i] > a[j] ) {  
        t = a[i];  
        a[i] = a[j];  
        a[j] = t;  
      }  
    }  
  }  
}
```

- Ignore the function definition
- Count operators and operands

3	<	3	{	1	0
5	=	3	}	2	1
1	>	1	+	1	2
1	-	2	++	6	a
2	,	2	for	8	i
9	;	2	if	7	j
4	(	1	int	3	n
4	)	1	return	3	t
6					

	Total	Unique
Operators	N1 = 50	n1 = 17
Operands	N2 = 30	n2 = 7

$V = 80 \log_2(24) \approx 392$
Inside the boundaries [20;1000]

Further Halstead metrics

	Total	Unique
Operators	N1	n1
Operands	N2	n2

Volume: $V = N \log_2 n$

Difficulty: $D = (n1/2) * (N2/n2)$

- Sources of difficulty: new operators and repeated operands
- Example: $17/2 * 30/7 \approx 36$

Effort: $E = V * D$

Time to understand/implement/test (sec): $T = E/18$

- Running example: 793 sec $\approx$ 13 min
- Does this correspond to your experience?

Bugs delivered: $E^{2/3}/3000$

- For C/C++: known to underapproximate • Running example: 0.19

Summary of Halstead metrics

Metric	Meaning	Mathematical Representation
n	Vocabulary	$n_1 + n_2$
N	Size	$N_1 + N_2$
V	Volume	$\text{Length} * \text{Log}_2 \text{Vocabulary}$
D	Difficulty	$(n_1/2) * (N_1/n_2)$
E	Efforts	$\text{Difficulty} * \text{Volume}$
B	Errors	$\text{Volume} / 3000$
T	Testing time	$\text{Time} = \text{Efforts} / S$, where $S=18$ seconds.

McCabe's complexity (1976)

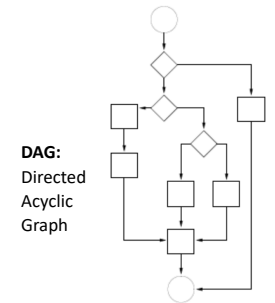
In general

- $v(G) = \#edges - \#vertices + 2$

For control flow graphs

- $v(G) = \#binaryDecisions + 1$, or

- $v(G) = \#IFs + \#LOOPS + 1$



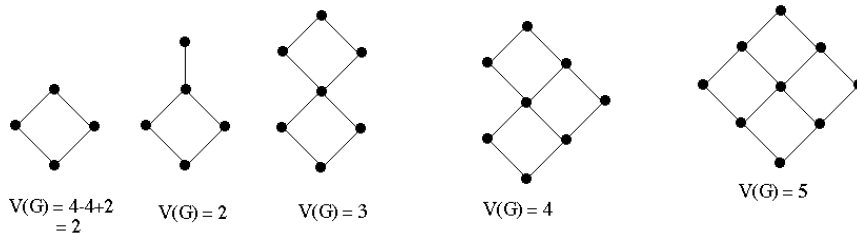
DAG:
Directed
Acyclic
Graph

Number of paths in the control flow graph. A.k.a. “**cyclomatic complexity**”

Each path should be tested!

$v(G)$ – is a **testability** metrics

McCabe's Cyclomatic Complexity

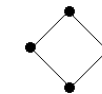


McCabe's Cyclomatic Complexity

```

if ( A < B ) then
    W;
else
    X;
endif
A = 3;
    
```

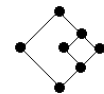
$V(G)=2$



```

if ( A < B ) then
    W;
else
    if ( B < C ) then
        X;
    else
        Z;
    endif
endif
    
```

$V(G)=3$



McCabe's complexity: Example

```
void sort ( int *a, int n ) {
    int i, j, t;

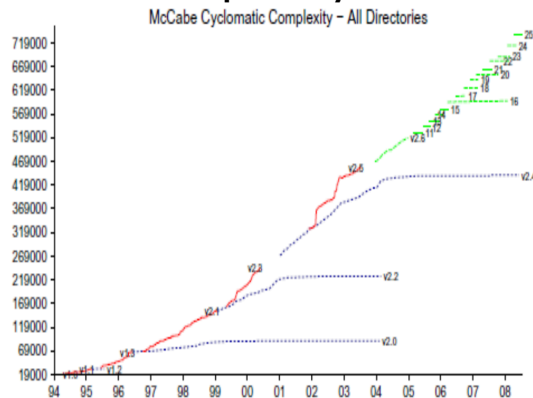
    if ( n < 2 ) return;
    for ( i=0 ; i < n-1; i++ ) {
        for ( j=i+1 ; j < n ; j++ ) {
            if ( a[i] > a[j] ) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}
```

- Count IFs and LOOPS
- IF: 2, LOOP: 2
- $v(G) = 5$
- Structural complexity

Questions for you

- Is it possible that the McCabe's complexity is higher than the number of possible execution paths in the program?
- Lower than this number?

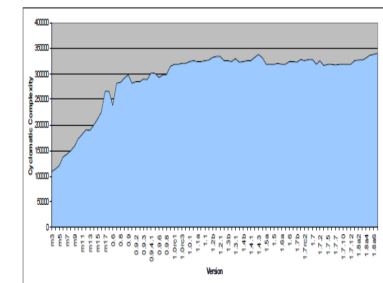
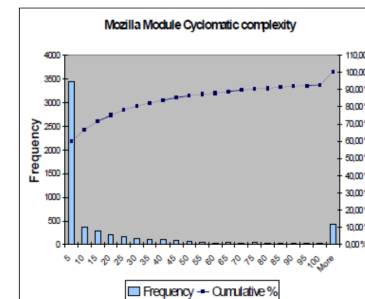
McCabe's complexity in Linux kernel



A. Israeli, D.G. Feitelson 2010

- Linux kernel
- Multiple versions and variants
- Production (blue dashed)
- Development (red)
- Current 2.6 (green)

McCabe's complexity in Mozilla [Røsdal 2005]



Most of the modules have low cyclomatic complexity
Complexity of the system seems to stabilize

Summarizing: Maintainability index (MI) [Coleman, Oman 1994]

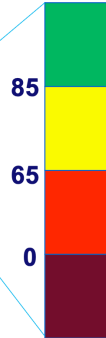
$$MI_1 = 171 - 5.2 \ln(V) - 0.23 V(g) - 16.2 \ln(LOC)$$

Halstead McCabe LOC

$$MI_2 = MI_1 + 50 \sin \sqrt{2.46 \text{ perCM}}$$

% comments

MI_2



- MI_2 can be used only if comments are meaningful
- If more than one module is considered – use average values for each one of the parameters
- Parameters were estimated mostly on large systems only

Chidamber & Kemerer OO Metric Suite

- WMC (Weighted Methods per Class)
- DIT (Depth of Inheritance Tree)
- NOC (Number of Children of a Class)
- CBO (Coupling Between Object Classes)
- RFC (Response for a Class)
- LCOM (Lack of Cohesion of Methods)

Metric 1: (WMC) Weighted Methods Per Class

Let C be a class, with methods $M_1, M_2, \dots, M_k$

Let $c_1, c_2, \dots, c_k$ be a complexity measure of the methods

Then WMC = $c_1 + c_2 + \dots + c_k$

indicates how much time and effort is required to develop and maintain the object

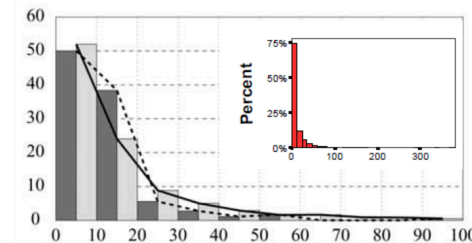
The larger WMC...

the greater the potential impact on the children

Objects are likely to be more application specific, limiting the possible reuse

WMC – weighted methods per class

- Sum of metrics(m) for all methods m in class C
- Popular metrics: McCabe's complexity and unity
- WMC/unity = number of methods
- Statistically significant correlation with the number of defects



- WMC/unity
- Gray: Mozilla 1.6
- Red: High- quality NASA system

Metric 2: (DIT) Depth of Inheritance Tree

The **depth of inheritance** of the class is the DIT metric for the class

The deeper a class is in the hierarchy, the more methods it is likely to inherit, making it more complex

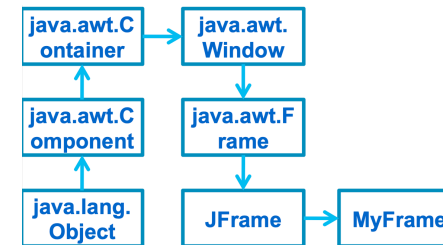
Deeper trees indicate greater design complexity (but also reuse)

A high DIT has been found to increase faults. A recommended DIT is 5 or less.

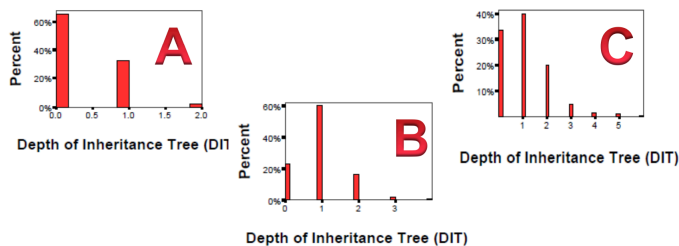
Depth of inheritance - DIT: Libraries?

Variants: Were to start and what classes to include?

- 1, JFrame is a library class, excluded
- 2, JFrame is a library class, included
- 7



DIT – what is good and what is bad?



Three NASA systems

- What can you say about the use of inheritance in systems A, B and C?
- Observation: quality assessment depends not just on one class but on the entire distribution

Metric 3: (NOC) Number of Children

NOC = the number of immediate sub-classes of the class

Depth is generally better than breadth in class hierarchy, since it promotes reuse of methods through inheritance

A high NOC can indicate several things:

- High reuse of base class. Inheritance is a form of reuse.
- Base class may require more testing.
- Improper abstraction of the parent class.
- Misuse of sub-classing. In such a case, it may be necessary to group related classes and introduce another level of inheritance.

High NOC has been found to indicate fewer faults. This may be due to high reuse, which is desirable.

Metric 4: (CBO) Coupling between Objects

CBO = number of classes to which a class is coupled

A is coupled to B if A uses methods/fields of B

$$\text{CBO}(A) = |\{B | A \text{ is coupled to } B\}|$$

Fan Out = # classes used by me

Fan In = # classes using me

$$\text{CBO} = \text{FanOut} - \text{FanIn}$$

High CBO is undesirable. Excessive coupling (>14) between object classes is detrimental to modular design and prevents reuse.

The larger the number of couples, the higher the sensitivity to changes in other parts of the design, and therefore maintenance is more difficult.

A high coupling has been found to indicate fault-proneness.

Metric 5: (RFC) Response for a Class

$$\text{RFC} = \text{NLM} + \text{NRM}$$

where:

NLM = number of local methods in the class

NRM = number of remote methods called by methods in the class

How much code (methods) is executed by a class in response to calls

A given remote method is counted only once

The larger the RFC figure:

testing and debugging becomes more complicated

the greater the complexity of the object

the harder it is to understand

Metric 6: (LCOM1) Lack of Cohesion in Methods

Chidamber and Kemerer (1993). LCOM or LOCOM or LCOM1

P = #pairs of distinct methods in C that do not share instance variables

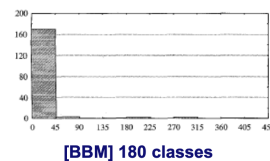
Q = #pairs of distinct methods in C that share instance variables

If $P > Q$ then

$$\text{LCOM} = P - Q$$

else

$$\text{LCOM} = 0$$



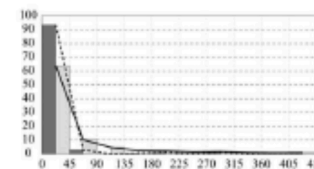
Discriminative ability is insufficient

What about methods that use get/set instead of direct access?

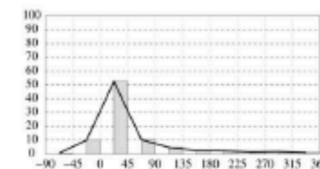
First solution: LCOMN

Defined similarly to LCOM but allows negative values

$$\text{LCOMN}(C) = P - Q$$

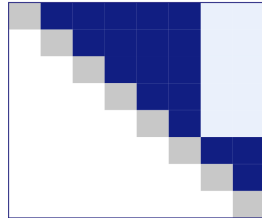
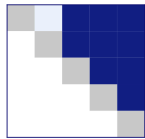


LCOM



LCOMN

Still...



LCOMN = P - Q

• **Method * method tables**

• Light blue: Q, dark blue: P

• **Calculate the LCOMs**

• **Does this correspond to your intuition?**

LCOM*: Henderson-Sellers, Constantine and Graham 1996

• **m – number of methods**

• **v – number of variables (attrs)**

• **m(V_i) - #methods that access V_i**

$$\frac{\left(\frac{1}{v} \sum_{i=1}^v m(V_i)\right) - m}{1 - m}$$

• **Cohesion is maximal: all methods access all variables**

$$m(V_i) = m \text{ and } LCOM = 0$$

• **No cohesion: every method accesses a unique variable**

$$m(V_i) = 1 \text{ and } LCOM = 1$$

Can LCOM exceed 1?

LCOM > 1?

If some variables are not accessed at all, then $m(V_i) = 0$

and if no variables are accessed

$$\frac{\left(\frac{1}{v} \sum_{i=1}^v m(V_i)\right) - m}{1 - m} = \frac{-m}{1 - m} = 1 + \frac{1}{m - 1}$$

Hence:

LCOM is undefined for m = 1

LCOM ≤ 2

Shortcomings of LCOM

A	Variables			
M e t h o d s	Dark	Dark	Light	Light
	Dark	Dark	Light	Light
	Light	Light	Dark	Dark
	Light	Light	Dark	Dark

B	Variables			
M e t h o d s	Dark	Dark	Light	Light
	Light	Dark	Dark	Light
	Light	Light	Dark	Dark
	Dark	Light	Light	Dark

C	Variables			
M e t h o d s	Dark	Light	Light	Light
	Dark	Dark	Dark	Light
	Light	Dark	Dark	Dark
	Light	Light	Light	Dark

Method-variable diagrams: dark spot = access

• **LCOM(A) ? LCOM(B) ? LCOM(C) ?**

$$\frac{\left(\frac{1}{v} \sum_{i=1}^v m(V_i)\right) - m}{1 - m}$$

Shortcomings of LCOM

A	Variables			
M et h o d s				

B	Variables			
M et h o d s				

C	Variables			
M et h o d s				

All LCOM values are the same: 0.67

$m=4, m(V_i) = 2$ for all i

A seems to be less cohesive than B and C !

$$\frac{\left(\frac{1}{v} \sum_{i=1}^v m(V_i)\right) - m}{1 - m}$$

Alternative LCOM [Hitz, Montazeri 1995]

LCOM as the number of strongly connected components in the following graph

Vertices: methods

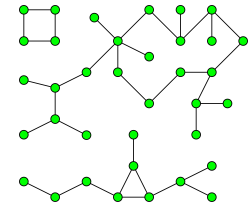
– except for getters/setters

Edge between a and b , if

– a and b access the same variable

• LCOM values

- 0, no methods
- 1, cohesive component
- 2 or more, lack of cohesion



Grafo con tre componenti totalmente connesse: Ogni nodo è raggiungibile da ogni altro nodo

NASA Object-Oriented systems study

NASA STUDY	System analyzed #1	System analyzed #2	System analyzed #3
Language	Java	Java	C++
Classes	46	1000	1617
Lines	50,000	300,000	500,000
Quality	"Low"	"High"	"Medium"
CBO	2.48	1.25	2.09
LCOM1	447.65	78.34	113.94
RFC	80.39	43.84	28.60
NOC	0.07	0.35	0.39
DIT	0.37	0.97	1.02
WMC	45.7	11.10	23.97

Laing, Victor & Coleman, Charles: Principal Components of Orthogonal Object-Oriented Metrics. White Paper Analyzing Results of NASA Object-Oriented Data. SATC, NASA, 2001. http://satc.gsfc.nasa.gov/support/OSMASAS_SEP01/Principal_Components_of_Orthogonal_Object_Oriented_Metrics.pdf

Metrics Case Study

Sharble & Cohen (1993)

Two OO designs of same system were studied

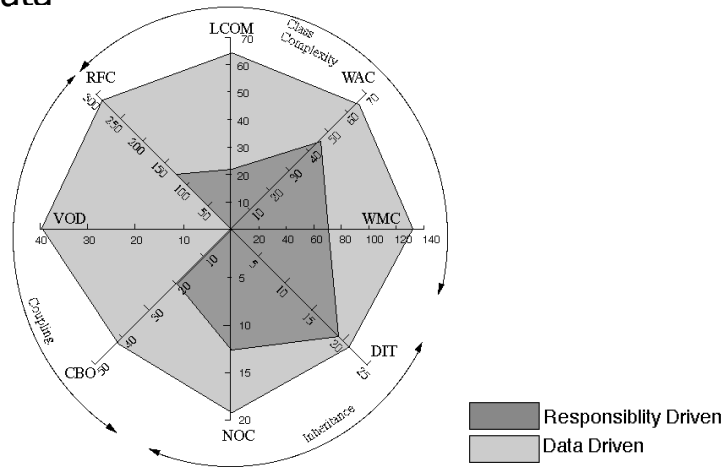
One design was produced using responsibility-driven methodology

One design using data-driven methodology

Data	Respon.		
Driven	Drive		
130	71	WMC	Weighted Methods Per Class
21	19	DIT	Depth of Inheritance Tree
19	13	NOC	Number of Children
42	20	CBO	Coupling between Objects
293	127	RFC	Response for a Class
64	21	LCOM	Lack of Cohesion in Methods
40	0	VOD	Violations of Demeter
64	44	WAC	Weighted Attributes per Class

Law of Demeter: use only one dot! YES: a calls $b.m()$ NO: a calls $b.c.m()$

Case Study Data



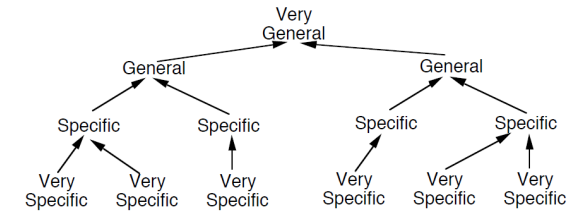
15 min. break



Chain of responsibility



Gerarchia di oggetti

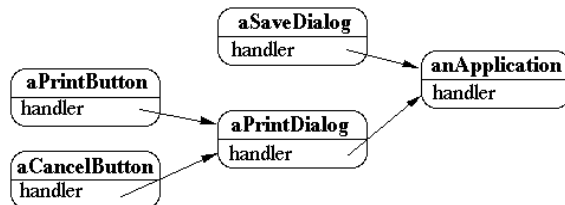


Gerarchia fissa, fatta con oggetti e non con classi:

Combina istanze di tipi non necessariamente correlati

Motivazione

Sistemi di Help Contestuale



l'oggetto che fornirà il messaggio d'aiuto **non è conosciuto dall'oggetto** (es. PrintButton) che inizia la richiesta

Delega ricorsiva

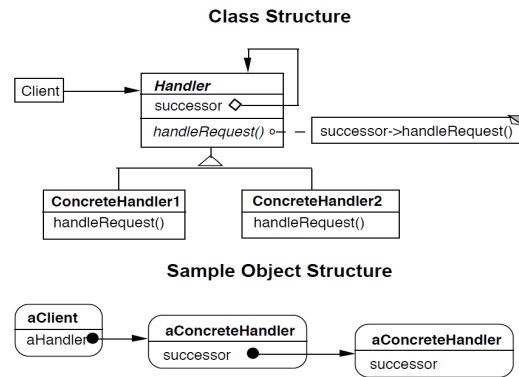
- Il pattern implementa di fatto una **ricorsione** orientata agli oggetti
 - La chiamata di un metodo corrisponde all'inoltro (polimorfico) della stessa chiamata al prossimo (diverso) ricevente
 - Alla fine una di queste chiamate corrisponderà all'esecuzione effettiva del metodo
 - La ricorsione quindi termina e si riavvolge all'indietro fino a tornare al chiamante originale.

CHAIN OF RESPONSIBILITY

- **Intento**

Dare la possibilità a più di un oggetto di gestire una richiesta.

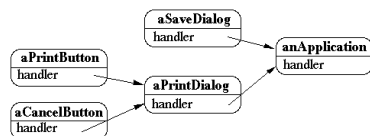
Disaccoppiare mandante e ricevente



Applicabilità

- Usa Il patter chain quando....
- Quando **più di un oggetto** potrebbe gestire una certa richiesta e non è noto a priori chi dovrà gestirla (*polimorfismo fuori gerarchia di classe*).
- Quando vuoi inoltrare una richiesta a un oggetto presente in un set di vari oggetti, **senza indicare il destinatario** esplicitamente (*accoppiamento lasco*)
- Quando il **set** di oggetti che può gestire una (o più) richieste deve essere specificabile/modificabile **dinamicamente**

Chain vs Decorator



Il primo oggetto attraversato che implementa il metodo richiesto, lo **esegue e interrompe** l'involo: **Un solo oggetto gestisce** la richiesta

La catena di oggetti **non è lineare** ma più genericamente ad **albero ed è prefissata**



Nel **decorator**, **tutti contribuiscono** in proprio alla esecuzione della richiesta

Non c'è un ordinamento prestabilito tra gli oggetti, anzi voglio proprio poterli combinare liberamente in varie sequenze

Definire la catena di responsabilità

- Posso usare riferimenti già esistenti tra gli oggetti
 - se ci sono già e corrispondono ad una gerarchia
 - se tale gerarchia coincide con una responsabilità crescente,
 - Tipico delle strutture dati composite (vedi il pattern *composite*)
- Altrimenti definisco appositi riferimenti aggiuntivi per i successori

Rappresentare le richieste: richieste “cablate”

```
abstract class HardCodedHandler {
    private HardCodedHandler successor;

    public HardCodedHandler(HardCodedHandler aSuccessor) {
        successor = aSuccessor;
    }

    public void handleOpen() {
        successor.handleOpen();
    }

    public void handleClose() {
        successor.handleClose();
    }

    public void handleNew(String fileName) {
        successor.handleClose(fileName);
    }
}
```

Ogni possibile richiesta ha una **segnatura** specifica ben precisa e **diversa dalle altre**.

Tutte le possibili richieste gestibili devono essere **raccolte nell'interfaccia** comune.

Il set di richieste è **noto a priori e fissato**

Rappresentare le richieste: singolo metodo parametrico (1/2)

```
abstract class SingleHandler {
    private SingleHandler successor;

    public SingleHandler(SingleHandler aSuccessor) {
        successor = aSuccessor;
    }

    public void handle(String request) {
        successor.handle(request);
    }
}
```

Il set di richieste non è fissato

Il parametro codifica le possibili richieste

Richiedente e riceventi prendono **accordi sulla codifica** delle richieste.

Rappresentare le richieste: singolo metodo parametrico (2/2)

```
class ConcreteOpenHandler extends SingleHandler {
    public void handle(String request) {
        switch (request) {
            case "Open": // do the right thing;
            case "Close": // more right things;
            case "New": // even more right things;
            default:
                successor.handle(request);
        }
    }
}
```

Nei concrete handlers...

Nuovi oggetti possono aggiungere supporto per codici addizionali corrispondenti a nuove richieste

Più controlli a runtime

Rappresentare le richieste: il parametro è la richiesta (1/2)

```
abstract class SingleHandler {
    private SingleHandler successor;

    public SingleHandler(SingleHandler aSuccessor) {
        successor = aSuccessor;
    }

    public void handle(Request data) {
        successor.handle(data);
    }
}

class ConcreteOpenHandler extends SingleHandler {
    public void handle(Open data) {
        // handle the open here
    }
}
```

Handler ha un solo metodo, con un parametro oggetto che **incapsula la richiesta**

Il set di richieste non è fissato

L'oggetto richiesta **contiene le info** che specificano la richiesta

La **esamino a run time** per capire il tipo di richiesta e, **se sono in grado**, processarla

possibile **sottoclassare** la richiesta e usare la **reflection** per esaminarla

Sempre controlli a runtime

Rappresentare le richieste: il parametro è la richiesta (2/2)

```
class Request {
    private int size;
    private String name;

    public Request(int mySize, String myName) {
        size = mySize;
        name = myName;
    }

    public int size() {
        return size;
    }

    public String name() {
        return name;
    }
}

class Open extends Request { // add Open specific stuff here }
class Close extends Request { // add Close specific stuff here }
```

Conseguenze

- **Riduce l'accoppiamento**
 - chi fa una richiesta non conosce il ricevente e viceversa
 - Un oggetto (handler) della catena non deve conoscerne la struttura
 - mantengono solo il riferimento al successore
- Si aggiunge **flessibilità** nella distribuzione di responsabilità agli oggetti
 - Si può cambiare o aggiungere responsabilità nella gestione di una richiesta cambiando la catena **a runtime**
- **Non c'è garanzia** che una richiesta venga gestita

Command

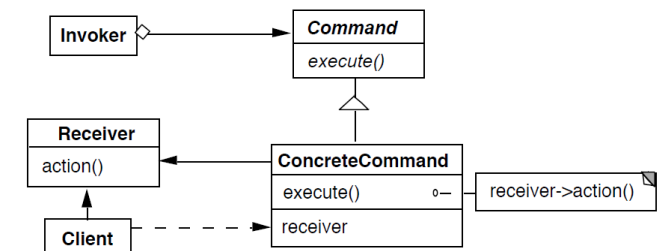
& Command Processor



Command

Intento:

- Incapsulare la richiesta di una operazione in un oggetto
- disaccoppia del tutto l'oggetto che invoca una operazione da quello che la esegue



È facile cambiare/aggiungere nuovi comandi, perché non devi cambiare le classi esistenti

Motivazione

Implementare Framework

Es.: Gestire un Menù

- Il framework implementa la classe *menù*
- non fissa le voci del menu né le corrispondenti azioni
- L'applicazione decide le voci e gli abbina le azioni
- È il set-up del framework

Implementazione (invoker)

```
class Menu {
    private Hashtable menuActions = new Hashtable();

    public void addMenuItem(String displayString,
        Command itemAction) {
        menuActions.put(displayString, itemAction);
    }

    public void handleEvent(String itemSelected) {
        Command runMe;
        runMe = (Command) menuActions.get(itemSelected);
        runMe.execute();
    }
    // lots of stuff missing
}
```

In una hash table abbino oggetti comando ad "azioni" simboliche (stringhe)

handleEvent (o ad esempio, l'**action listener** di un oggetto awt/swing su cui ho cliccato) **seleziona e invoca l'istanza** di comando ad essa abbinata in corrispondenza

Implementazione (concrete command)

```
abstract class Command {
    abstract public void execute();
}

class OpenCommand extends Command {
    private Application opener;

    public OpenCommand(Application theOpener) {
        opener = theOpener;
    }

    public void execute() {
        String documentName = AskUserSomeHow();
        if (name != null) {
            Document toOpen = new Document(documentName);
            opener.add(toOpen);
            opener.open();
        }
    }
}
```

In questo caso, il **ricevitore** è una applicazione di word processing.

Attraverso la classe (framework) Menù, riceverà la richiesta di aprire un certo documento.

Il concrete command incapsula tale richiesta e viene a sua volta incapsulato (set-up) nel Menu.

Applicabilità

- Uso il pattern command:
 - Invece di definire funzioni callback staticamente: incapsulo l'azione in un parametro *Comando*, *gestibile a run-time*
 - Per avere in **tempi diversi** la **definizione**, l'**invocazione** e l'**esecuzione** di comandi
 - L'oggetto command può **differire l'esecuzione** dell'azione. Ad esempio, accumularle e poi eseguirle a lotti.
 - Il command object può elaborare dati relativi alle richieste ricevute e, **poiché ha il receiver**, anche **ispezionare il risultato** di quelle eseguite
 - Per **creare comandi di alto livello** assemblando insieme sequenze di azioni in una azione composita complessa (MACRO)

Conseguenze

- **Flessibilità nell'invocazione** delle richieste
 - Gli stessi comandi possono essere generati da elementi diversi (ad es. dell'interfaccia utente)
 - L'utente stesso potrebbe configurare le azioni corrispondenti agli elementi di interfaccia
- **Flessibilità nel numero e nella funzionalità** delle richieste
 - **Aggiungere nuovi** comandi o definire macro è facile
- **Flessibilità nell'esecuzione** delle richieste
 - I comandi possono essere conservati per ripeterli in seguito
 - Possono essere loggati
 - Possono essere annullati
 - Possono essere eseguiti in parallelo su threads separati

Svantaggi

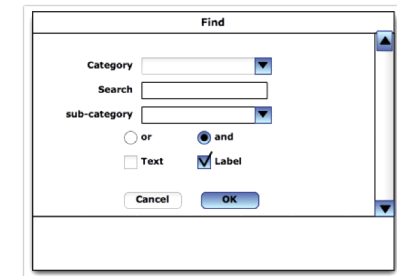
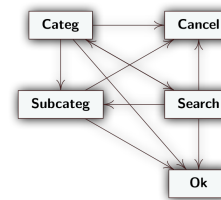
- Leggera perdita di efficienza per l'indirezione
- Potenziale perdita di efficienza per l'aumento delle classi
 - Evitabile usando pluggable command
- Aumento della complessità
 - Non è certo nell'invoker l'effetto dell'invocazione attivata

Mediator

Object behavioural



Motivazione



- **Ogni elemento** visualizzato (testo, lista, bottone, etc.) è controllato da una corrispondente **classe**
- Ciascuna classe deve **comunicare il suo stato** alle altre per far aggiornare la **visualizzazione**
- Ciascuna classe (senza un Mediator) chiamerà i metodi di tutte le altre classi, e quindi **ciascuna classe è dipendente dalle altre**

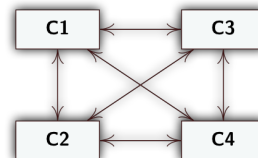
Mediator

• Intento

- Definire una interfaccia per la comunicazione tra un gruppo di oggetti che interagisce.

• Motivazione

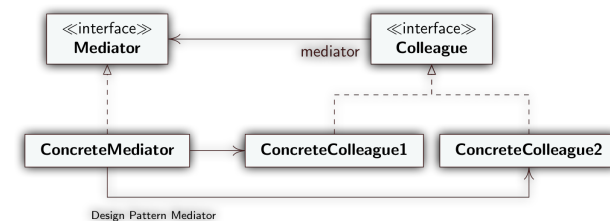
- **Riuso:** Se ho molte interazioni (dipendenze) l'intero sistema si comporta come se fosse monolitico.
- **Estensione:** Diventa difficile cambiare il comportamento anche dell'intero sistema poiché è distribuito fra gli oggetti e dovrei sottoclassarne molti o tutti.



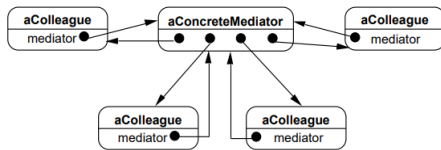
Mediator

• Soluzione

- Isolare le comunicazioni (dipendenze) tra oggetti (cooperanti) in un sottosistema, creando una classe dedicata Mediator



Mediator: Struttura a run-time



- L'oggetto mediator è responsabile per la gestione (**istanzia e mantiene i riferimenti**) ed il **coordinamento** dell'interazione tra i colleghi: invoca i loro servizi su loro richiesta e/o iniziativa propria
- Non è una facciata rivolta all'esterno (facade) : è ad uso interno, un direttore che dirige i suoi musicisti
- Façade non aggiunge funzionalità; mediator ha invece al suo interno **tutta la logica di funzionamento** che prima era distribuita e può incorporarne **anche di nuova**
- I colleghi sono **consapevoli** del mediator, con cui interagiscono; i componenti del sottosistema sono inconsapevoli dell'esistenza del Façade. Per i client non cambia nulla in entrambi i casi

Esempio: COOLING SYSTEM

- a simple cooling system consists of a fan, a power supply, and a button.
- Pressing the button will either turn on or turn off the fan.
- Before we turn the fan on, we need to turn on the power.
- Similarly, we have to turn off the power right after the fan is turned off.
- Tre component interdipendenti: **Fan, PowerSupply e Button.**

Varianti

- Il concrete mediator accede alle interfacce di tutti i concrete colleagues direttamente,
- Ma i colleagues come accedono al mediator?
 - Mediator con **metodi espliciti**: corrispondenti a quelli nei colleague
 - Mediator con **metodo unico generico**: se serve, passo anche il colleague per identificare chi lo invoca
 - Metodo **generico in overloading**: uno per ogni tipo di concrete colleague
- **Cooling**: il mediator **riporta** esplicitamente **metodi dei colleagues** che ora sono invocati dai colleagues su di lui

Metodo generico unico

```
class DialogDirector {
    private Button button;
    private Fan fan;
    private PowerSupply powerSupply;

    // altro codice...

    public void doWork( Colleague me ) {
        if ( me == button ) blah
        else if ( me == fan ) more blah
        else if ( me == powerSupply ) even more blah
    }
}
```

Metodo generico in overload

```
class DialogDirector {
    private Button button;
    private Fan fan;
    private PowerSupply powerSupply;
    private Button anotherButton;

    // altro codice

    public void doWork(Button b) {
        if ( b == button ) // Process button task
        else if ( b == otherButton ) // Process other button task
    }
    public void doWork(Fan f) {
        // Process fan task
    }
    public void doWork(PowerSupply b) {
        // Process PowerSupply task
    }
}
```

Mediator

- Applicabilità
 - Usare il Mediator quando
 - Un insieme di oggetti comunicano in modo **ben definito ma intricato**. Le interdipendenze che ne risultano sono non strutturate e difficili da comprendere
 - **Riusare un oggetto** è difficile poiché esso comunica con tanti altri oggetti
 - Un comportamento che è distribuito fra tante classi dovrebbe essere **modificabile senza dover ricorrere a sottoclassi**: lo centralizzo nel ConcreteMediator e cambio solo lui.

Considerazioni finali

- La maggior parte della **complessità** che risulta nella gestione delle dipendenze è **spostata** dagli oggetti cooperanti **al Mediator**. Questo rende gli oggetti più facili da implementare e mantenere
- Le classi Colleague sono più riusabili poiché la loro **funzionalità** fondamentale **non è mischiata con** il codice che gestisce le **dipendenze**
- Il codice del **Mediator non è in genere riusabile** poiché la gestione delle dipendenze implementata è solo per una specifica applicazione: può essere direttamente una **classe concreta**
- Mediator è una astrazione proprio per consentire eventualmente di cambiarne implementazione a parità di colleagues